Monitoring and Profiling of AI Agents

Anonymous (for review)

***Abstract*—When an AI agent causes harm—whether through adversarial prompt injection or benign mis-execution—incident responders must reconstruct which actor invoked which tool, under which privileges, and where data moved. Under TLS 1.3, HTTP/2, and QUIC/HTTP/3, network-layer monitoring can show destinations, timing, and transfer metadata, but it does not by itself show actor binding, tool-invocation causality, context lineage, or policy-decision records needed for audit-ready reconstruction.**

To support post-incident reconstruction of agent sessions under encrypted and multiplexed transports, the paper derives minimum practical requirements (MPR) for agent telemetry across five observation and enforcement points, six telemetry axes, and seven normalized event types. The framework specifies the minimum identity, correlation, tool-path, policy-decision, and egress signals needed to support session-scoped post-incident reconstruction and audit-ready attribution, along with the join keys and operational conditions for deterministic cross-source reconstruction across five deployment architectures.

As a scoped visibility assessment, an IDE-based proxy corpus spanning four recorded runs and 8,161 raw request-response wrappers was then assessed against the derived MPR baseline using only directly intercepted request-response artifacts. In that directly intercepted view, the corpus exposes recurring control-plane and setup surfaces, but leaves context-lineage records, policy-decision records, approval-workflow evidence, and stable cross-source join propagation unevidenced.

The paper contributes a practical, survey-backed MPR framework, expressed as a schema-level event model, for evaluating agent-security claims, designing audit-ready telemetry pipelines, and comparing solutions by joinability rather than feature lists. It is not a new detection system or benchmark.

***Index Terms*—agentic AI security; observability; telemetry; MCP; tracing; semantic gap; network visibility; workload identity**

Use of AI tools. AI-based tools were used to assist literature discovery, manuscript organization, language refinement, and early-stage wording alternatives for conceptual framing. All research questions, framework commitments, evidentiary judgments, citation checks, and technical conclusions were independently determined, reviewed, and finalized by the authors. The authors reviewed, verified, and take full responsibility for the content of this publication.

I. Introduction

A. Problem Background: The Semantic Gap in Network Visibility

In agentic AI systems, the security objective extends beyond output inspection. Incident response and governance require reconstruction of which execution actor invoked which tool under which privileges, and where data moved. In early triage, identical side effects (e.g., destructive tool actions or sensitive egress) can arise from attacker-controlled inputs or from benign agent error.

Audit-ready telemetry must therefore support evidence-based attribution and cause discrimination, rather than relying on assumptions about adversarial presence. What matters here is the cross-system execution path, not an isolated model response.

Among the available instrumentation surfaces in enterprise environments, network-boundary monitoring is the most commonly deployed and therefore a natural starting point for assessing how much of this execution path can be captured. In boundary-visible enterprise deployments where traffic already passes through firewalls, proxies, and gateways, network-boundary monitoring is a natural first candidate for capturing part of this execution path. Under modern transports, however, boundary-only monitoring centered on L3/L4 signals is structurally insufficient for such reconstruction.

TLS 1.3 encrypts most handshake and application details [1]. HTTP/2 multiplexes concurrent requests over one connection [2]. QUIC provides an encrypted, multiplexed transport substrate [3]. HTTP/3 extends application semantics over that transport [4]. ECH can further reduce destination-level inference from handshake metadata [5].

Consequently, boundary telemetry is often limited to coarse metadata (e.g., IP, port, protocol, timing, and volume). Under these transport conditions, the execution-level signals needed for reconstruction—tool-call parameters, context provenance and sensitivity, policy-decision outputs, approval-workflow evidence (where separately instrumented), and egress effects—remain outside direct network visibility. Threat taxonomies and agentic guidance recognize the security relevance of logging, monitoring, lineage, and traceability for LLM and agent incidents [6], [7], and broader governance frameworks reinforce monitoring requirements for AI systems [8]. However, the minimum field set and cross-source join conditions for audit-ready reconstruction remain unspecified.

B. Research Questions and Scope

As the related-work comparison in Section II substantiates, existing work addresses transport-limit analysis, tracing and correlation standards, and security controls around tool paths, but it does not combine them into a scoped minimum practical requirements (MPR) framework for agent incident reconstruction under the forensic questions and deployment assumptions used here. The framework developed in this paper therefore begins by fixing the relevant observation and enforcement surfaces, trust boundaries, and bypass conditions. The three research questions below define that scope:

- **RQ1 (Sections II–III):** Across enterprise environments, where do the key observation and enforcement points (EP1–EP5) sit across the endpoint/runtime, identity,

model-interaction, tool-path, and egress layers, what trust boundaries do they define, and what bypass surfaces remain?

- **RQ2 (Section IV; Appendix A):** When heterogeneous telemetry is merged into a session- or incident-level reconstruction view, which fields and correlation keys recur across standards and vendor artifacts, and which fields must be promoted to REQUIRED when recurrence alone is insufficient for the scoped tasks?
- **RQ3 (Section V):** In a scoped IDE-based proxy corpus spanning multiple recorded runs, which schema-relevant signals become directly visible, and which evidence classes remain absent in the intercepted direct-evidence view?

Within this scope, the empirical component is intentionally narrow and is limited to the visibility findings from the intercepted case study in Section V; it does not quantify performance, operational overhead, or deployment prevalence.

Despite this empirical narrowness, the framework scope retains both adversarial incidents and non-adversarial agent misexecution because their side effects can be indistinguishable at triage time, especially for attribution, tool-path causality, context lineage, and evidence reliability.

The requirement set is intended as a sufficient minimum only for the scoped forensic questions used here—session-scoped post-incident reconstruction and audit-ready attribution. It is bounded by the deployment categories and incident assumptions formalized later in Section II rather than serving as a universal question set for all agent-security evaluation tasks.

Out of scope are detection-model benchmarking, training-time attack analysis (e.g., poisoning/fine-tuning attacks), and development of new jailbreak or prevention techniques. These exclusions reflect the study's focus on audit-ready telemetry for post-incident reconstruction and diagnosis rather than on upstream prevention or detection-model performance. Privacy, regulation, and legal accountability are treated as secondary constraints: the telemetry model assumes data minimization and purpose limitation, with collection restricted to minimum fields and correlation keys and with de-identification (e.g., summarization, hashing, masking) applied where needed.

C. Summary of Contributions

The contributions are:

- A characterization of the semantic visibility gap created by encrypted and multiplexed transports.
- A derived minimum practical requirements set for session-scoped post-incident reconstruction and audit-ready attribution, obtained through artifact-based requirement inference from the scoped forensic questions. Where recurrence alone is insufficient, Appendix E justifies REQUIRED-field promotion through the scoped substitute test.
- A schema-level formalization of that requirement set as six telemetry axes (T1–T6) with an interoperable schema and event taxonomy, separating recurrence-derived fields from those promoted by scoped non-substitutability.
- A bounded visibility assessment of an IDE-based proxy corpus against T1–T6 and EP1–EP5, identifying directly visible signals and residual gaps.
- An explicit treatment of non-adversarial agent misexecution—whose side effects can be indistinguishable from adversarial steering at incident time—as an additional driver for the same telemetry requirements.

Intended audience: These contributions are intended primarily for enterprise security architects, SOC analysts, and incident-response teams who need to design or evaluate audit-ready telemetry pipelines for agentic AI deployments. The work is also relevant to platform engineers integrating agent runtimes with identity, gateway, and observability infrastructure, and to researchers studying agent security, observability, or tool-use governance.

Paper guide: Table I summarizes the recurring internal labels used in the sections that follow; the detailed scenario labels S1–S13 are defined in Appendix D and referenced selectively in the main text. General technical acronyms are expanded at first use; the most frequent are MCP (Model Context Protocol), NHI (non-human identity), MITM (man-in-the-middle), and ECH (Encrypted ClientHello). The next section fixes the working definitions, identity preconditions, trust boundaries, observation model, and network-only limits that ground the later survey, framework, and case study.

TABLE I
SUMMARY OF RECURRING LABELS.

Label set	Meaning
EP1–EP5	Observation / enforcement points along the agent execution path
BP1–BP5	Deployment categories (local, bypass, enterprise, hybrid, delegated SaaS)
T1–T6	Telemetry axes (attribution, tool-path, context, egress exposure, policy, integrity)
E1–E7	Normalized event types (session, model I/O, context, tool, identity, policy, egress)
FQ1–FQ6	Scoped forensic questions
R1–R5	System-level requirements
M1–M4	Evaluation dimensions
S/D/X	Evidence grades (Schema/Spec / Documented / Not verifiable)
R/N/R+N/C	REQUIRED-field promotion basis (recurrence / necessity / both / conditional)

II. Background and Assumptions

This section sets out the conceptual and architectural conditions for agent incident reconstruction from telemetry—working definitions, identity preconditions, threat model, observation model, and network-only limits—along with the scoped related-work position that motivates the framework formalized later.

A. Definitions

The key working definitions are as follows. *Agentic AI* denotes a capability pattern that iterates goal-directed planning, execution, observation, and revision. *AI agent* denotes an operational system that combines model inference with context construction (e.g., retrieval and memory), tool execution (e.g., function calling or the Model Context Protocol, MCP), privilege and approval handling, and audit logging.

When such an agent becomes the subject of post-incident analysis, the central task is to build a faithful record of what it did and why. Profiling here means constructing a session-scoped execution profile from telemetry—that is, a structured record of a single agent run that captures enough audit-relevant detail for post-incident reconstruction. It does not mean deriving a long-horizon behavioral baseline or an organizational risk rating.

Specifically, this profile captures the minimum audit-relevant attributes needed to interpret one agent run as an auditable execution actor: actor binding, session/request lineage, tool-path actions, context lineage, policy-decision outputs, approval-workflow evidence (where separately instrumented), and egress effects.

The scope is therefore a single-run profile. Persistent behavioral modeling or risk-tier assignment may build on repeated profiles, but those downstream analytics are out of scope.

Among these profile elements, policy-decision outputs are treated as the baseline policy-layer signal; approval-workflow evidence is analyzed only where the deployment exposes a separate approval step. Both elements are formalized in the framework's telemetry-axis definitions (Section III-C).

B. Non-Human Identity (NHI) and Workload Identity: Technical Preconditions

The definitions above identify actor binding and session lineage as core profile elements. Both depend on a prerequisite: reliable attribution requires a stable non-human identity (NHI) at workload runtime. NHI here means a cryptographically verifiable workload identity with short-lived credentials that can be validated through mTLS or signed-token flows—the technical precondition for answering "who executed this action" at incident time.

SPIFFE defines workload identifiers and SVID identity documents [9]. SPIRE performs node/workload attestation to issue and verify SVIDs [10]. SPIFFE overview material documents rotation support at the ecosystem level [11]. Together they provide the reference workload-identity model used here.

With that identity foundation in place, telemetry from different observation points can in principle be joined at session or request granularity—linking actor attribution to tool invocations and egress events through shared correlation keys. The exact field classes and join conditions for this correlation are derived in Section IV; the remainder of this subsection defines the identifier levels on which those joins depend.

a) Identifier levels (normative): Four identifier levels are distinguished: workload, session, request, and conditional lineage.

`agent_id` denotes a workload-level non-human identity that should remain stable across sessions for the same execution actor. `session_id` denotes an interaction/run-level context that may span multiple tool executors and gateways. `trace_id`/`span_id` (or `request_id` as fallback) denotes a request/operation-level correlation anchor.

The fourth level, conditional lineage, applies to graph-structured execution: deterministic child-causality reconstruction additionally requires explicit parent-child lineage fields such as `parent_span_id` or `parent_request_id`. The request/operation anchor alone is insufficient once sibling or delegated child operations coexist.

In BP5-like delegated environments where only vendor identifiers are exposed, logs MUST mark identifier scope (enterprise-scoped vs. vendor-scoped) to prevent false attribution across tenants or runtimes.

Scope-preservation note: Canonical field-family alignment in Tables VII and A2 is used for recurrence accounting and comparison only. It does not imply value-level interchangeability between cryptographically verified workload identities (e.g., SPIFFE IDs) and application-level agent identifiers (e.g., `gen_ai.agent.id`), or between client/application sessions (`session.id`) and conversation/thread identifiers (`gen_ai.conversation.id`). Normalized records MUST preserve original scope qualifiers.

C. Threat Model and Trust Boundaries

With actor binding established, the threat model focuses on the incident classes and trust boundaries that determine which evidence is needed and where it must be collected.

The threat model centers on confused-deputy execution: an agent with formally valid privileges can be induced to misuse that authority [12]. The baseline assumption is that legitimate application authority can be misused through crafted prompts [13], [14]. The same formally valid application authority can also be misused through retrieved context [15]. Memory or knowledge-base poisoning is a related variant that can steer later planning and execution [16]. These prompt-, context-, or memory-driven steering effects can then propagate into downstream tool or API use [13], [17].

The same cause-discrimination problem—determining whether a harmful outcome resulted from adversarial steering or from benign agent error—applies when no external attacker is involved: an agent may mis-execute under formally valid privileges and produce harmful side effects that are observationally similar to adversarial steering.

Non-adversarial incident driver (F0): Enterprises also face incidents where no external attacker is present, but the agent misinterprets benign instructions, hallucinates state, or produces erroneous plans and then executes harmful tool actions under formally valid privileges. Because the resulting side effects can be indistinguishable from adversarial steering, the same telemetry axes—especially T2 (tool-invocation causality), T3 (context lineage), and T6 (evidence reliability)—are required for evidence-based discrimination.

Both the adversarial and the non-adversarial drivers operate over the same set of protected assets and actor categories. Protected assets include internal code, documents, tickets, secrets, and delegation-chain credentials. We model three attacker classes: (A1) direct or indirect prompt-injection operators, (A2) malicious or compromised tool/connector endpoints, and (A3) insiders or misconfigured privileged users. Separately, we retain F0 as a non-adversarial incident driver in which harmful actions arise from agent mis-execution without an external attacker.

The analysis anchors evidence collection on four trust boundaries: (i) agent runtime to external model/tool services, (ii) token issuer to token consumer, (iii) egress boundary (proxy/SSE/gateway), and (iv) long-term evidence storage (e.g., SIEM/WORM). These boundaries identify where T1–T6 evidence must be collected to reconstruct causality and impact. The corresponding scoped forensic questions are formalized later in Section III-C under "Scoped forensic questions (FQ1–FQ6)."

D. Observation Model: EPs (Observation and Enforcement Points) and Correlation Keys

At each trust boundary, the question is: what signals can be collected, and how are they linked? The five observation and enforcement points below map the trust boundaries from Section II-C to concrete evidence surfaces along the agent execution path.

Observation and enforcement locations are abstracted as five EPs that map to the trust boundaries above: EP1 (endpoint/runtime) and EP2 (identity/privilege) cover the runtime and identity surfaces; EP3 (model interaction/policy) covers the model boundary; EP4 (tool invocation path) covers the tool boundary; and EP5 (egress/data boundary) covers the egress boundary. Evidence storage remains a separate cross-cutting integrity requirement (T6).

Each EP emits a distinct evidence class: EP1 host/process/runtime clues, EP2 token and delegation events, EP3 model I/O and policy outcomes, EP4 tool-invocation sequences, and EP5 destination/volume plus DLP/blocking events. An EP may be observation-only or enforcement-capable, depending on deployment control over the relevant trust boundary.

For session-level reconstruction, the baseline linear-join key set is (`agent_id`, `session_id`, `trace_id`/`span_id` or `request_id`). Among these, `trace_id`/`span_id` are application-layer identifiers propagated across components [18], [19], while `agent_id`, `session_id`, and `request_id` are framework-defined or deployment-derived anchors not inferred from flow metadata. When the execution path includes fan-out, repeated tool calls, or delegated runtimes, this baseline is extended conditionally by `parent_span_id` or `parent_request_id`, `tool.call_id`, and `delegation.chain_id`.

Unlike the application-layer event types E1–E6 described in Table I, E7 (network/flow events) operates under a different join model and uses two conformance modes—a network-observable profile and a join-baseline profile, formalized in Section IV. Join-capable records carry session/request keys for deterministic cross-EP joins, while network-observable records are aligned secondarily through 5-tuple and NAT translation logs, optionally augmented by host/process-to-socket mapping. TLS 1.3 encrypts application data and reduces directly visible protocol semantics at the boundary [1]. QUIC provides the encrypted transport context for HTTP/3 [3]. HTTP/2 multiplexes multiple exchanges within one connection [2]. HTTP/3 carries comparable application semantics over a QUIC transport context [4]. In this framework, those transport properties mean that a 5-tuple alone does not preserve request-level boundaries.

Among these five observation and enforcement points, EP4 (tool invocation path) is particularly protocol-dependent in practice. The EP4 abstraction itself is protocol-agnostic; MCP is treated as one concrete protocol surface in the survey (Section III-B3) and case study (Section V). Where claims about tool-path observation or enforcement are made, they rely on separate artifact types rather than on the base protocol description alone: MCP security guidance [20], transport authorization materials [21], and gateway observability artifacts [22].

E. Limits of Network-Only Observation and Man-in-the-Middle (MITM) Interception

The observation model described so far relies on application-layer telemetry. Network-layer interception — via TLS-terminating proxies or similar MITM setups — might appear to offer an alternative, but in practice it cannot serve as the default.

Operational designs cannot assume MITM decryption as a default capability. Client trust of the interception CA constrains TLS interception in practice [66]. Certificate pinning and related controls add a separate practical constraint [66], [67]. QUIC/HTTP/3 is therefore treated here as a separate transport path that must be handled explicitly when interpreting interception results.

MITM is an optional upper-bound measurement posture, not a universal deployment prerequisite.

What matters for reconstruction, then, is not the interception mechanism itself but the preservation of cross-EP joinability: the ability to map host/process context to network flows and to propagate request-level identifiers (e.g., trace_id/request_id) across components. Organizations can in principle satisfy this with different instrumentation combinations, including endpoint telemetry, runtime/EDR logs, application instrumentation, and gateway or service-mesh telemetry. Section IV formalizes the exact field sets and join conditions required.

F. Differentiation from Prior Academic Work (Related Work)

Given those observation limits, the next question is what prior academic work already covers and where this study adds scoped coverage. Three bodies of literature are most relevant.

1) Agentic threats and prompt manipulation: Empirical studies show that prompt injection can manipulate application behavior and induce unintended actions in LLM-integrated applications [13]. In RAG pipelines, indirect prompt injection can succeed end-to-end across retrieval and generation stages [15]. Memory or knowledge-base poisoning is an important threat [16]. In this study, that threat further motivates telemetry for context provenance and integrity (T3). Additional analyses emphasize layered controls and monitoring rather than filter-only prevention assumptions [14], [17]. This study complements those attack-focused studies by specifying infrastructure-level observability and correlation requirements.

2) Network visibility limits and "semantic gap" framing: A second body of work concerns the transport constraints noted in Section I-A—TLS 1.3 encryption [1], HTTP/2 multiplexing [2], QUIC/HTTP/3 transport encryption [3], [4], and ECH [5]—and their implications for network-only telemetry under modern deployment conditions. These properties serve as architectural constraints that motivate a minimum application- and gateway-layer telemetry set rather than as deficiencies of specific products.

3) Tracing, provenance, correlation, and schema standardization: Finally, cross-system correlation and reproducible reconstruction are most naturally expressed in tracing and structured-log models. W3C Trace Context defines interoperable correlation identifiers [18]. OpenTelemetry provides structured telemetry models and correlation guidance for instrumented systems [19]. For agentic systems, emerging GenAI semantic conventions indicate a path for standardizing model- and tool-level telemetry [39]. GenAI metrics conventions add complementary quantitative telemetry [40]. Closely related provenance and distributed-forensics work also informs this area: PROV-DM supports provenance representation and interchange [43]; Dapper supports distributed tracing and correlation [44]; CamFlow illustrates provenance capture and integration [45]; and CamQuery illustrates runtime analysis over captured provenance [46].

Those standards and systems supply lineage or tracing mechanisms but do not specify the minimum cross-EP field sets and join conditions needed for tool-path attribution, policy-decision evidence, and egress-exposure accounting under the scoped forensic questions addressed here.

4) Positioning of this work: Table II summarizes the comparison axes used in this section. The table delimits scope against prior work and motivates why the proposed contribution is a technology-agnostic framework for audit-ready telemetry: (i) where signals are observed (EP1–EP5), (ii) what is recorded (T1–T6, E1–E7), and (iii) how heterogeneous signals are normalized and joined. The novelty claim is not that provenance or tracing is invented anew. Rather, those foundational ideas are composed into that framework for agent incident reconstruction, under the deployment and evidence assumptions stated in Sections II–IV.

Read together, the rows of Table II show that existing work addresses important subsets of these columns — transport limits, correlation mechanisms, or tool-path security — but, under the scoped problem formulation used here, no single prior framework combines transport limits, cross-system correlation, tool-path security, and scoped minimum requirements for agent incident reconstruction.

The related-work comparison therefore delimits the bounded contribution of this study: composing those strands into a technology-agnostic MPR framework under the deployment and evidence assumptions stated above. The next section then surveys the commercial defense ecosystem to show where current segments observe, what they can verify, and which telemetry gaps still remain.

III. Deep Survey of the Commercial Defense Ecosystem Centered on EPs

Together with Section II, this section completes the answer to RQ1 by locating EP1–EP5 across enterprise trust boundaries and by identifying the bypass surfaces that remain. It compares segments by where they observe, what can be verified there, and which reconstruction gaps remain. The recurring gaps identified through that comparison are then distilled into the six MPR axes (T1–T6) and scoped forensic questions (FQ1–FQ6) used in the framework.

A. Access Principles and Evidence Strength

A survey based on public documentation cannot directly observe internal implementations. Accordingly, this section does not treat undocumented capability claims as evidence. It compares solutions by whether the events and fields needed for incident reconstruction are publicly verifiable through official specifications, documentation, and log/event interfaces.

The unit of comparison is not product names but which telemetry is emitted at which trust boundary; the conclusions are presented as minimum requirements for multi-layer correlation rather than as vendor rankings.

1) Survey Scope and Data Collection: Survey targets are grouped into five segment families: (i) the model-interaction boundary (guardrails/content safety), (ii) tool-path control (MCP/tool gateways), (iii) inline blocking at the network/runtime layer, (iv) DLP and compliance, and (v) observability and tracing (vendor-neutral). To evaluate "auditability" in each segment, the survey prioritizes official protocols and standards, official specifications and security guides, provider or gateway audit-log/event descriptions, OpenTelemetry semantic conventions and log models, and—where available—concrete event-schema examples (JSON fields, parameters, record structures). The citations that follow are methodology examples of those source classes rather than proofs of the survey method itself: transport standards [1]; protocol and security documentation [20], [23]; log models [19]; tracing standards [18]; and GenAI semantic conventions [39].

The corpus contains 53 primary survey artifacts spanning standards, government frameworks, vendor or organization documentation, and open-source specifications. Appendix A additionally documents the supplementary specifications used for field/key canonicalization and for the Table A2 evidence matrix, so the appendix source list remains complete for all

TABLE II
COMPACT RELATED-WORK COMPARISON BY COVERAGE AXIS.

Area (examples)	L4–L7 limits	Tracing / correlation	Tool-path security	Minimum practical requirements (MPR)
Encrypted transport limits [1], [2], [3], [4], [5]	✓			
Agent/LLM threat studies and guidance (selected tool-path-risk context) [6], [7], [13], [15], [16], [17]			✓	
Workload identity primitives [9], [10], [11]		✓		
Tracing/GenAI conventions [18], [19], [39], [40]		✓		
Provenance / distributed forensics [43], [44], [45], [46]		✓		
MCP tool-path governance [20], [21], [23]		✓	✓	
Proposed framework	✓	✓	✓	✓

cited references. Access dates are recorded in the reference list, and artifacts were prioritized when they exposed machine-verifiable log/event schemas over marketing-only claims (Table III).

2) Evidence-Strength Grades (S/D/X): Table III establishes the grading rule used throughout this section and later in Tables V and VI. Appendix A carries the field-level evidence matrix (Table A2) and the supporting source list (Table A1), but the main body uses the grades more narrowly: S-grade artifacts support field semantics directly, whereas D-grade artifacts are used only to establish signal presence without extending undocumented field meanings.

Throughout Section III, X-grade indicates non-verifiability in public artifacts rather than the absence of a capability; such cases are phrased as "not publicly verifiable."

3) Artifact-Based Requirement Inference: A signal, as used here, is a publicly specified, machine-verifiable event or field (e.g., log field name, JSON schema attribute, protocol header) that can be mapped to (i) an EP emission/enforcement point and (ii) at least one telemetry axis (T1–T6). Narrative claims without an observable output interface are classified as X-grade.

The inference extracts only publicly verifiable signals and assigns an evidence grade (S/D/X). It then maps each signal to one or more EPs according to where the signal is emitted or enforced and to one or more telemetry axes according to what the signal contributes to incident reconstruction. The recurring gaps and join requirements exposed by that coding then motivate the minimum event model in Section IV. Included artifacts are limited to official standards, official vendor or organization documentation that specifies or describes audit logs, APIs, authorization flows, tracing/metrics, or policy enforcement, and open-source documentation that defines an instrumentation surface; marketing-only material, inaccessible proprietary portals, and undocumented capability claims are excluded.

TABLE III
EVIDENCE-STRENGTH GRADES FOR PUBLIC-ARTIFACT-BASED COMPARISON.

Grade	Definition (verification criteria)
S (Schema/Spec)	Official specifications/standards or official documentation explicitly define the structure of logs/events (field names, JSON schema, API parameters, etc.).
D (Documented)	Official documentation states that an event/log is generated and explains its meaning, but provides limited field-level schema detail.
X (Not verifiable)	Public artifacts do not allow verification of the signal/field (or access is not available); only marketing claims exist.

Coding records both location and function. EP assignment follows where the documented signal is emitted or enforced (endpoint, identity provider, gateway, tool path, or egress boundary), while T1–T6 assignment follows what the signal enables for incident reconstruction: attribution (T1), tool invocation (T2), context lineage (T3), egress exposure (T4), policy-decision outputs and approval-path evidence where separately instrumented (T5), and evidence integrity, retention, and access audit (T6). A single signal may support more than one telemetry axis, but the coding records one primary axis plus any secondary contributions rather than forcing a 1:1 assignment.

When artifacts disagree, the analysis prefers S over D over X and prefers standards or protocol specifications over vendor summaries when the disagreement concerns cross-system correlation or transport semantics. Ambiguous mappings are downgraded rather than resolved by inference because no independent inter-rater validation was available. Appendix A records the detailed validation note and the fixed promotion rubric used for Table A2, including alias canonicalization limits, recurrence support, scoped substitute testing, and the R/N/R+N/C basis labels.

4) Deployment-Scenario Lens (BP1–BP5, S1–S13): Even with the same "AI security" strategy, observability and enforceability still depend on deployment structure. In BP1 (no network), EP5 controls are inherently ineffective, whereas in BP5 (fully SaaS/external runtime delegation) EP1/EP4 signals may be unavailable at the enterprise boundary and the upper bound is set by vendor-provided logs. The comparison therefore uses the deployment taxonomy below—defined by tool-execution location, inference-engine location, routing path, and asset authorization—as an interpretive lens rather than assuming a single environment.

Table IV fixes BP1–BP5 as the active comparative lens for the segment comparison that follows, while S1–S13 remains appendix detail for scenario-level decomposition.

Accordingly, the gaps discussed below are treated as consequences of deployment structure and trust-boundary placement, not as universal failures of any one solution category.

TABLE IV
DEPLOYMENT TAXONOMY (BP1–BP5) AND TYPICAL OBSERVABILITY BREAKS.

Category	Description (high level)	Typical observability breaks
BP1: Fully local / zero-network	Inference and tools run on a local host with minimal external calls.	Network gateways see little or no traffic; requires endpoint telemetry for tool actions.
BP2: Edge inference / shadow IT bypass	Inference or tools run on unmanaged endpoints; traffic may bypass enterprise controls.	Observation gaps due to out-of-band routing and unmanaged authorization state.
BP3: Enterprise-controlled (on-prem / dedicated tenant)	Inference and tool execution are within enterprise-managed infrastructure.	Typically broader coverage for EP3/EP4; identity and trace propagation remain necessary across services.
BP4: Hybrid with external inference	Enterprise tools combine with third-party model APIs.	EP5 sees encrypted traffic and limited semantics; attribution relies on endpoint/gateway instrumentation.
BP5: Delegated SaaS runtime	Agent runtime executes inside third-party SaaS; enterprise is a client.	Limited internal visibility; must rely on vendor logs, contractual telemetry, and coarse egress controls.

B. Segment Comparison Based on EP1–EP5

Within the deployment-and-trust-boundary frame established above, the five segments below are strongest at different trust boundaries. EP5 (egress)-centric approaches are strong on destinations and transfer volumes (part of T4), but tool-invocation events (T2) and context provenance/sensitivity (T3) can remain structural gaps. Conversely, EP4 (tool path) gateways make it easier to structure tool-call sequences, but local file/IDE context and non-MCP calls require separate instrumentation.

The key comparison dimensions are not feature names but (i) at which trust boundary a signal is generated, (ii) how far bypass paths can be closed, and (iii) whether signals can be correlated with other EPs through propagation of correlation keys.

1) Segment Comparison Table (Summary): Table V summarizes major segments in terms of "strong EPs" and "structural gaps." In the table, "strong EP" indicates the choke point at which a strategy primarily observes and enforces; the auxiliary EPs in parentheses are points that are often combined in real deployments. Across the five segments, the same contrast recurs: each layer is strongest at one boundary, leaves another part of reconstruction unresolved, and depends on adjacent controls for join-complete incident reconstruction.

Table V therefore shows that, among the five surveyed segment families, no single solution category provides complete audit-ready reconstruction on the basis of publicly verifiable artifacts alone. EP4-centered controls are strongest for tool-path activity (T2) and action-level allow/deny records, whereas EP5-centered controls are strongest for egress visibility and parts of the policy path (T4/T5). The segment readings below therefore treat each layer as a partial view that must hand off to adjacent controls through shared join keys.

2) AI Gateway / Provider Guardrails (EP3 with EP5 as auxiliary): AI gateways and guardrail services sit at the model-interaction boundary, so they see model I/O and model-side policy outcomes most directly. Provider guardrail services can analyze model inputs and, where documented, enforce model-I/O controls [25], [26], [37]. Where product documentation exposes request-level policy-assessment outputs or logs, those outputs should be cited separately. For example, Model Armor logging can include evaluation results and additional metadata [61]. Azure Prompt Shields returns prompt-attack analysis results [62]. Bedrock Guardrails returns `action`, `actionReason`, `assessments`, and `usage` in guardrail responses [63]. Token- or request-volume telemetry should likewise be cited separately from policy outcomes. Bedrock documents token-count metrics [64]. AI Gateway request logs can expose token-usage records [65].

The limitation is that tool execution often occurs after the model call in separate runtimes or connectors. The AI-gateway/guardrail layer therefore cannot fully reconstruct downstream tool chains (T2) or context provenance/lineage (T3) across external systems.

Because this layer is strongest on model-side policy outputs, it is most useful when the incident turns on policy over model I/O; once downstream actions matter, the evidence must be completed by tool-path and egress controls.

3) MCP Gateway / Tool-path Control (EP4 with EP2 as auxiliary): Tool-path control sees the next boundary directly. When all tool invocations traverse a single mediated path and the gateway emits the relevant records, MCP gateways can document tool name, parameters, and results as a structured event sequence (T2) [22]. Because allow or deny can be enforced at the tool or parameter level, this layer is also well-suited to explicitly separating and auditing high-risk tools (e.g., data export, external HTTP calls, file writes) (T5).

The protocol surface underlying this segment is MCP, introduced publicly as an open protocol and ecosystem for tool and data connectivity [32]. Separately, MCP security guidance emphasizes constraining server-side capabilities and explicitly hardening tool and server boundaries [20]. The MCP authorization specification defines an OAuth-based flow for HTTP transports and distinguishes STDIO transports that obtain credentials from the environment [21]. OAuth deployment and token handling should follow best current practices, including mitigations for authorization-code and token threats [33]. Some commercial MCP gateway implementations integrate OAuth2-based authorization to mediate access to MCP servers [34].

The central weakness, however, is architectural bypass. From a threat-model perspective, the first question is whether tool calls truly always pass through EP4. Execution can still occur through non-MCP paths, and MCP-specific token-passthrough or local-server patterns can further weaken EP4 mediation [20].

TABLE V
SEGMENT COMPARISON CENTERED ON EPS (TELEMETRY PERSPECTIVE, NOT PRODUCT NAMES).

Observability strategy	Strong EP	Verifiable primary artifacts (examples)	Main telemetry produced (examples)	Structural gaps (typical)
AI Gateway / Provider Guardrails	EP3 (+EP5)	Cloud guardrails/content-safety documentation [25], [26], [37]. Product-specific policy-output examples include Model Armor sanitize-operation logging [61], Prompt Shields analysis outputs [62], and Bedrock Guardrails response fields [63]. Separately documented token/request telemetry examples remain distinct [64], [65]. The retained T4/T5 grouping in this row is framework-defined rather than a single source-native schema.	Model-I/O control surfaces; documented policy-assessment outputs (T5); where exposed, token/request metadata (part of T4).	Tool-call chains (T2) and context provenance (T3) are difficult to cover end-to-end across external systems.
MCP Gateway / Tool-path Control	EP4 (+EP2)	MCP security guidance [20]. MCP authorization materials [21]. Gateway-observability documentation [22]. These sources supply separate security, authorization, and observability primitives; the retained T2/T5 grouping in this row is framework-defined rather than a single source-native schema.	Tool-call events (T2); action-level allow/deny (T5); where exposed, server/client identifiers (part of T1).	Non-MCP calls that bypass the MCP path; local file/IDE context (T3).
Runtime Firewall / Network Intercept	EP3/EP5 (+part of EP4)	Inline detection/blocking documentation [28].	Inline detection/blocking events (T5); egress metadata (T4); limited protocol threat signals.	L7 semantics are limited under encryption, pinning, and QUIC; precise attribution (T1) requires coupling with IdP/endpoint data.
SSE/DLP & Compliance	EP5 (+part of EP3)	DLP/compliance documentation [29], [30]. Destination-field semantics [53]. Byte-volume semantics [55]. Request logging where exposed [65]. Provider token-count metrics where exposed [64]. The retained T4/T5/T6 grouping in this row is framework-defined rather than a single documented product schema.	DLP detec-tion/masking/blocking (T5); destination/volume evidence (T4); compliance audit events where documented (part of T6).	Internal agent runtime/tool invocations (T2) and data provenance (T3) are difficult to observe directly.
Tracing/Observability (vendor-neutral)	EP3–EP4 (primary)	Tracing/observability specifications/documentation [19], [31]. GenAI semantic and span conventions [39], [59]. Metrics conventions [40].	Correlation primitives (T1); model I/O events; tool-call spans (T2); extensible event hooks that can carry policy-decision payloads only when separately instrumented (T5).	Enforcement (blocking or human approval) requires a separate policy engine or workflow; collection depends on instrumentation.

Note: In this table, structural gaps denote architecture-level observation limits of the segment itself, whereas non-publicly-verifiable gaps in the surrounding segment analysis denote signals that may exist operationally but are not evidenced by the present public-artifact corpus.

EP4 strategies consequently need companion controls: egress or runtime controls that can block or detect non-MCP calls (EP5/EP1), coupling with token issuance and usage events (EP2), and session correlation through propagated trace or request identifiers.

In comparative terms, EP4 sharpens action-chain visibility but still depends on runtime and egress controls when execution can bypass the mediated path.

4) Runtime Firewall / Network Intercept (primarily EP3/EP5): Runtime firewalls and inline network interception see the transport boundary most directly. They use network chokepoints to detect and block selected patterns (T5) [28], contributing meaningful transport-boundary enforcement.

Their main limitation is protocol visibility. TLS 1.3 encrypts handshake and application elements [1]. HTTP/2 multiplexes concurrent exchanges on one connection [2]. QUIC provides the encrypted transport context [3]. HTTP/3 carries HTTP semantics over that transport [4]. Where deployed, ECH can further reduce destination-metadata visibility [5]. Under those conditions, the inline layer may be limited to destination, volume, timing, and narrow signature-based signals (part of T4).

Precise attribution still requires coupling with IdP or workload-identity evidence (EP2) and endpoint execution clues (EP1). In practice, this segment is best understood as transport-boundary enforcement that must be paired with higher-layer correlation rather than as a self-sufficient reconstruction layer.

5) SSE/DLP & Compliance (primarily EP5): Where runtime firewalls and inline interceptors enforce at the transport level, SSE, DLP, and compliance solutions address the egress boundary most directly. SSE, DLP, and compliance solutions are optimized for compliance oversight [30]. Some deployed products also support controlling data exfiltration or screened

egress [29]. Where documented, they can provide policy-enforcement outcomes such as pattern matching and flag or block actions [29]. Where request logging is exposed, they can surface boundary metadata relevant to T4 [65]. Standardized destination semantics are grounded separately in network-attribute sources [53]. Byte-volume semantics are grounded separately in flow-export sources [55]. Adjacent retention guidance can inform retention-oriented aspects of evidence handling [38]. Any broader connection to T6 in this framework remains only a partial evidence-control contribution, not a source-native audit schema.

Based on publicly verifiable artifacts, this layer lacks explicit schemas that directly identify which tool call triggered the exfiltration (T2) or from which files, rules, or memories the context was assembled (T3).

For DLP events to support incident reconstruction, they must remain joinable with event sequences from other EPs through compatible actor, session, and request-level keys (for example, `agent_id`, `session_id`, and `trace_id`/`span_id` or `request_id`) where those identifiers are actually emitted and propagated. Otherwise, they indicate mainly that something left the boundary. Without those joins, egress-centric controls remain boundary alerts rather than reconstructive evidence linked back to earlier EPs.

6) Tracing/Observability (vendor-neutral, coupling EP3–EP4): Tracing and observability are the correlation layer across the gateway, tool-path, runtime, and egress segments described above rather than a standalone security control. Their main role is to supply correlation primitives rather than a complete audit-ready join rule. W3C Trace Context defines interoperable correlation identifiers [18]. OpenTelemetry provides the carrier and correlation substrate via `trace_id`/`span_id` [19]. For agentic systems, generative-AI semantic conventions extend that substrate toward model I/O events [39]. GenAI span conventions extend it toward tool-invocation traces [59]. GenAI metrics conventions add complementary quantitative usage telemetry [40].

Using those tracing primitives together with separate identity-binding primitives, this framework derives two main advantages. First, when compatible identifiers are emitted and propagated across the participating EPs, logs produced at EP3 (model interaction), EP4 (tool calls), and EP5 (egress) can be joined into one correlated event sequence. Trace Context and OpenTelemetry provide the tracing primitives [18], [19]. Separate workload-identity and owner-binding primitives come from SPIFFE/SPIRE and OIDC materials [9], [10], [11], [50]. The joinability conclusion itself is this framework's synthesis rather than a precomposed rule from any one source. Accountable T1 attribution still depends on separate identity binding rather than tracing alone. Second, standardizing a minimum field set can reduce vendor lock-in, because a common correlation schema allows organizations to substitute or add products without losing cross-EP join capability.

The main limitation is that tracing provides observation, not enforcement. Blocking decisions and any separately instrumented approval-workflow evidence (T5) still require separate policy engines and gates. Because collection depends on instrumentation, the upper bound in some deployment environments (e.g., BP5) is set by what the vendor exposes.

Gateway implementations such as LiteLLM expose configuration settings for proxy logging and callbacks, including OpenTelemetry export paths [35]. Open-source instrumenters such as OpenLLMetry provide OpenTelemetry-compatible traces for LLM calls and related application or framework activity [36]. Tracing therefore contributes the join fabric across the preceding segments, but it still depends on those segments to supply the actions, policy outcomes, and boundary effects being joined.

7) Summary: EP Combination Patterns and Deployment-Specific Priorities: Interpreting the five segments through the deployment taxonomy yields deployment-relative priorities for which EPs must be secured first. In BP1 (no network), EP5-based control is impossible, so EP1 (endpoint/runtime) and EP2 (identity/tokens), along with local audit logs, are central. In BP2 (out-of-band bypass/shadow), inference can complete outside the enterprise observability zone, so endpoint authorization (Auth) and token issuance or usage events (EP2) must at least secure attribution (T1) and post-incident investigability. In BP3 (enterprise controlled), EP4 (tool path) and EP5 (egress) can be made chokepoints, making it practical to combine tool-invocation sequences (T2) and egress-control events (T4/T5) at high resolution.

In BP4 (external inference), policy decisions inside the model move outside the enterprise boundary, so it is effective to accept policy or guardrail outputs (T5) provided at EP3 while keeping tool execution enforcement at EP4. In BP5 (full delegation), securing EP1 or EP4 at the enterprise boundary is difficult, and vendor logs set the upper bound.

Although the priority and achievable coverage of each EP varies across BP1–BP5, the same evidence categories—attribution, tool-path activity, context lineage, egress exposure, policy/approval-path evidence, and evidence integrity, retention, and access audit—recur as structural gaps across deployment structures. Because these categories recur across deployment-specific priorities rather than appearing in only one configuration, they constitute cross-cutting evidence needs rather than artifacts of a particular deployment choice. The minimum practical requirements are therefore not the union of all survey gaps, but the least evidence set needed to answer the scoped forensic questions under the stated evidence and deployment assumptions. They are introduced in two steps: the next subsection distills the retained set into six cross-deployment telemetry axes (T1–T6) and six scoped forensic questions (FQ1–FQ6), and Section IV then formalizes them into a normative event model. The non-substitutability of each resulting axis is verified through the failure-mode analysis in Appendix E.

C. Survey Distillation into Cross-Deployment Telemetry Axes

Using the recurring cross-deployment gap categories identified above, this section formalizes audit-ready incident reconstruction as six scoped telemetry axes: attribution (T1),

tool-invocation causality (T2), context lineage (T3), egress exposure accounting (T4), policy-decision outputs with separately instrumented approval evidence where available (T5), and evidence integrity, retention, and access audit (T6). Here "minimum" names the least signal set needed to answer the scoped forensic questions below under the stated evidence and deployment assumptions.[1]

a) Scoped forensic questions (FQ1–FQ6): FQ1–FQ6 constitute the minimum forensic question set used here for session-scoped post-incident reconstruction and audit-ready attribution. FQ1–FQ5 ask what happened along the execution path, while FQ6 asks whether those answers remain evidentially trustworthy.

- **FQ1 (Accountable attribution):** For each externally observable effect (e.g., egress or policy decision), can we attribute it to a specific executing agent/workload, owner boundary, and session with evidentiary confidence?
- **FQ2 (Causal chain reconstruction):** What is the end-to-end chain from initiating input/context through model and tool actions to observed side effects, and which action step caused which effect?
- **FQ3 (Data lineage and available sensitivity context):** Which inputs and execution-context elements influenced the action? Relevant examples include internal artifacts, retrieved context, prompt-like inputs, tool descriptors or schemas, model/runtime context when it materially shapes action interpretation, and delegated-path lineage where relevant. What lineage links connect those inputs to outputs and effects, and, where a deployment classifies them, what available sensitivity attributes qualify those inputs or context elements?
- **FQ4 (Policy/approval path evidence):** Can we demonstrate the policy-decision path for an action and, where a deployment inserts a separate human approval step, the approval path and timing relative to the action?
- **FQ5 (Exposure accounting and blast radius):** Can we characterize exposure scope and blast radius with enough fidelity for triage and impact analysis by relating observed effects back to responsible actions and sessions? Operationally, this question is instantiated using destination class, time window, and magnitude indicators (e.g., bytes/tokens) where available.
- **FQ6 (Evidence reliability):** As a cross-cutting evidentiary question, can the collected records be treated as reliable incident evidence (integrity, retention, and access-audit history), and are evidentiary gaps detectable?

These forensic questions are reconstruction questions, not attack-detection questions; they apply regardless of whether an external attacker is present. Even in a non-adversarial failure, distinguishing benign agent error from attacker-controlled steering still depends on reconstructing the tool-action chain (FQ2/T2), enumerating context lineage (FQ3/T3), and relying on tamper-evident evidence controls (FQ6/T6). Section II-C centers on confused-deputy steering only for threat-model concreteness.

To make those abstract questions operational, the following illustrative (non-empirical) worked incident shows what each forensic question would require in a concrete reconstruction task.

Consider a coding agent embedded in an IDE that is instructed by a developer to repair failing tests in a repository. A repository documentation artifact contains hidden instructions that steer the agent to read a secret-bearing `.env` file and an internal design document, then send selected contents to an external URL through an HTTP-capable or MCP-connected tool. At the boundary, the visible outcome may be only an outbound request and a proxy or DLP alert. The reconstruction task is therefore not merely to confirm that external transmission occurred, but to determine which execution actor and session produced it, which action chain led to it, which inputs shaped it, how policy controls treated it, what exposure followed, and whether the available records are trustworthy enough to support those conclusions.

The scenario sharpens six distinct investigative demands:

1) **FQ1:** The investigator should be able to identify the accountable execution actor: which agent or workload, under which owner boundary, in which session, produced the outbound transfer. It should therefore go beyond a generic user-account reference and support a statement such as: "the outbound transfer was produced by coding-agent session S running in user A's IDE environment." Without FQ1, concurrent agent runs, background tasks, or shared services remain difficult to separate.
2) **FQ2:** The record should reconstruct the causal chain from the initiating developer instruction, through repository-context ingestion and sensitive-file access, to tool invocation and outbound transmission. In this incident, the answer may take the following form: after the developer's repair instruction, the agent interpreted hidden instructions in repository documentation, read the `.env` file and internal design material, invoked an external-transfer-capable tool, and sent a request to an external URL. The causal chain captured by FQ2 narrows the set of candidate explanations, but full discrimination between attacker-controlled steering and non-adversarial agent mis-execution additionally requires the input-lineage evidence addressed by FQ3; without both, the incident narrative remains incomplete.
3) **FQ3:** The record should identify the inputs and execution-context elements that shaped the action. In this scenario, it may take the following form: the agent's behavior was shaped by hidden instructions in the repository documentation artifact, values read from the `.env` file, and context recovered from the internal design document; where the deployment classifies them, the `.env` contents and design-document fragments would additionally carry elevated sensitivity. The question therefore does not stop at listing what was read; it asks which inputs materially shaped the action and whether

[1]Here "minimum" is used in an engineering, scope-relative sense, not as a formal complexity-theoretic result.

that contribution arose from attacker-controlled context or from non-adversarial execution context. This FQ3 answer is the data-lineage complement to the sequence-level answer required by FQ2.

4) **FQ4:** The investigator should be able to establish how policy or approval mechanisms treated the action: whether the external-transfer tool path was allowed, denied, or modified (e.g., redacted or brokered), and whether any separate human approval step existed before or after the attempted action. In this incident, the answer may take the form either that the transfer path was allowed without a separate approval step or that a deny decision blocked the outbound action. Without FQ4, the incident may be visible while the governance path remains unclear.
5) **FQ5:** The record should account for destination, timing, transfer volume, exposed material, and blast radius across one or more sessions, rather than stopping at the weaker conclusion that data left the boundary. In this incident, the answer may take the form that approximately Y bytes were sent to external domain X, including secret-bearing values from the `.env` file and fragments of internal design material, and that the currently observed blast radius is limited to one user session. Exposure triage and impact analysis remain incomplete without this additional accounting.
6) **FQ6:** The evidence set must show whether the agent trace, proxy logs, policy records, and related evidence-control records are sufficiently complete and trustworthy to support the preceding answers with evidentiary confidence. In this incident, the answer may take the form either that agent, proxy, and policy logs align in time and correlation keys and therefore support high evidentiary confidence, or that a missing tool trace prevents a strong conclusion. This is why FQ6 remains a cross-cutting evidentiary question rather than one more content-layer question.

Read operationally, the scenario shows that each forensic demand depends on a distinct evidence class that must remain joinable across EP boundaries. Those demands map directly onto the six telemetry axes below:

- T1 anchors the accountable actor, owner boundary, and session needed for FQ1.
- T2 records the action-chain evidence needed to reconstruct the causal path in FQ2.
- T3 is motivated in part by provenance framing such as PROV-DM [43]. In this framework, T3 preserves the input, context-lineage, and available sensitivity context needed for FQ3.
- T4 captures destination, timing, and transfer volume for the exposure accounting required by FQ5.
- T5 records policy decisions and approval-workflow evidence needed for FQ4.
- T6 provides the framework's trustworthiness bundle for FQ6. Retention-policy controls [38], tamper-evident records [41], and audit-record and access-control guidance [42] motivate separate adjacent primitives relevant to T6. The combined bundle itself is a framework-defined composition rather than a recurrent source-native schema.

Each axis is therefore non-substitutable for at least one scoped forensic question: removing it either materially degrades that question or makes it unanswerable under the current incident, evidentiary, and deployment assumptions. Figure 2, Appendix E, and Table E1 formalize the corresponding axis-removal and substitute-signal analysis, while Table VIII remains the normative per-event conformance checklist.

Figure 2 formalizes the worked incident and its telemetry consequences as a matrix linking FQ1–FQ6, T1–T6, and the REQUIRED field sets used in Table VIII; it also makes explicit that FQ6 operates as a cross-cutting trust condition over the other questions rather than as one more content-layer event class.

1) Network-Only Fillability and Its Limits: Table VI tests whether any telemetry axis can be satisfied at the network boundary alone. The answer explains why Section IV formalizes a cross-EP minimum schema rather than a network-only one. Request-level semantics are difficult to reconstruct from the network boundary (L3/L4) alone, and L7 meaning—such as tool-call parameters, context provenance, policy-decision outputs, and approval-workflow evidence (where separately instrumented)—remains a structural gap.

As Table VI shows, the network can provide only part of T4 (destination, volume, and timing), while T1–T3 and T5–T6 remain structural gaps without events from the application, privilege, and policy layers. Even for T4, the partial fillability is constrained: HTTP/2 multiplexes multiple request/response exchanges within one connection [2]. QUIC provides the encrypted transport context [3], and HTTP/3 carries comparable request semantics over that transport [4]. Because connection-level identifiers are insufficient in multiplexed transports, deployments should propagate L7 correlation identifiers such as trace_id/request_id. W3C Trace Context provides a standard way to do so [18].

2) Design Implication: Reduction to a Common Event Model: The survey and MPR analysis therefore show that audit-ready execution reconstruction is fundamentally a cross-EP correlation problem rather than a product-selection problem.

The minimum telemetry target is therefore not a single product log format but a common event model that can normalize heterogeneous signals into one incident event sequence. The cross-deployment reduction completed here identifies the shared evidence classes and scoped forensic questions that survive across deployment structures; the next section turns those findings into the normative event model, schema, and conformance rules.

IV. Minimum Practical Requirements and Framework Proposal (Core Framework)

Where Section III-C distills the retained axes and scoped forensic questions, this section answers RQ2 by formalizing

TABLE VI
MINIMUM PRACTICAL REQUIREMENTS (T1–T6) AND FILLABILITY WITH NETWORK-ONLY SIGNALS.

Telemetry axis	Primary EP(s)	Verifiable official basis (examples)	Fillable via network-only signals?
T1 Actor attribution	EP1/EP2/EP3	SPIFFE/SPIRE workload-identity primitives [9], [10], [11]. OIDC issuer/subject claims for owner binding [50]. Trace Context and OTel log correlation for cross-system joins [18], [19]. The retained T1 row is a framework-defined join bundle over those separate primitives rather than a single source-native schema.	No
T2 Tool invocation/execution (invocation trace)	EP4 (+EP3)	MCP security guidance [20]. MCP authorization [21]. Gateway observability [22]. The retained T2 row is a framework-defined interoperability bundle over those separate tool-path and request-logging primitives rather than a single source-native schema.	No
T3 Context provenance/sensitivity (lineage)	EP4 (+EP3)	MCP tool-path security guidance for the EP4 control surface [20]. Provenance-model framing for lineage semantics [43]. Trace or log-correlation materials for correlation carriers [19], [59]. The exact T3 field family, including sensitivity classification, is framework-defined rather than source-native.	No
T4 Egress metadata/exposure volume	EP5	Cloudflare AI Gateway DLP feature docs for boundary scanning [29]. Request logging docs for exposed request metadata [65]. OTel network attributes for destination fields [53]. IPFIX octet-count elements for byte-volume fields [55]. Provider token-count metrics where exposed [64]. The combined T4 egress field family remains framework-normalized rather than a single documented product schema.	Partially
T5 Policy-path evidence (policy-decision outputs; approval workflow where separately instrumented)	EP3/EP4/EP5	Model Armor sanitize-operation logging [61]. Prompt Shields analysis API [62]. Bedrock Guardrails response outputs [63]. These sources directly support product-specific policy or guardrail outputs. The recurrent policy-output family used here is framework-normalized rather than a source-native common schema, and approval-workflow evidence remains a deployment-specific extension rather than a directly recurrent surveyed field set.	No
T6 Integrity/retention/access audit (evidence chain)	Cross-cutting (SIEM/storage)	Retention-policy controls [38]. Tamper-evident log validation [41]. Audit-record content guidance [42]. The exact T6 evidence-control bundle retained in this row is framework-defined rather than a recurrent source-native schema.	No

them into a minimum event and field contract for cross-EP reconstruction. It first defines five system-level requirements (R1–R5), then introduces the event taxonomy and correlation keys (E1–E7), and finally specifies the schema and operating conditions needed for deterministic cross-EP reconstruction under the stated assumptions.

A. System Requirements

The coverage gaps identified in Section III (Tables V and VI) motivate five system requirements (R1–R5) for reproducible incident reconstruction.

R1 (*Coverage*): Deployments must emit at least one verifiable event stream for each telemetry axis T1–T5 and a verifiable cross-cutting control-record set for T6.

R2 (*Attribution*): Deployments must preserve stable actor binding through `agent_id`/owner and identity-layer evidence.

R3 (*Correlation*): W3C Trace Context and OTel log correlation provide the underlying correlation primitives [18], [19]. On that basis, deployments should propagate shared session or request correlation keys across EPs.

R4 (*Decision auditability*): General audit-event selection and audit-record-content guidance such as AU-2 and AU-3 motivate the need for auditable security-relevant records [42]. Within this framework, deployments must preserve policy-decision outputs together with decision identifiers and timestamps. The decision-record preservation rule is therefore a framework requirement motivated by that guidance rather than a field contract mandated by any single source. If a deployment includes a human approval step, the corresponding approval-workflow evidence must also be logged as a correlated extension.

R5 (*Purpose limitation*): Deployments must support minimum-necessary collection so that joins remain feasible without default retention of raw content.

R5 operational safeguards: R5 adds three safeguards. First, telemetry pipelines MUST NOT persist raw bearer tokens, session cookies, API keys, or full authorization headers. Second, export paths MUST apply deterministic redaction or scrubbing before storage or forwarding. Third, when content linkage is operationally necessary, this framework prefers keyed digests with recorded algorithm metadata and a verifiable key reference over unsalted hashes.

When keyed digests are used, the schema requires `digest.key_ref` to reference the key version used for digest generation, and the investigation window to retain the corresponding key-management audit records. If a digest key is rotated or compromised, investigators MUST be able to identify the affected time range and downgrade evidentiary confidence for that range.

Beyond those data-handling safeguards, R5 also has an operational-cost implication. Higher telemetry fidelity increases retention cost, normalization cost, and instrumentation overhead (latency and logging volume). Prior tracing/provenance work highlights low-overhead instrumentation [44] and selective or tailored processing [45], [46]. In this framework, those results motivate the operational tradeoff noted here rather than a fixed deployment-cost formula.

While R1–R5 define the operational requirements, one additional cross-cutting element is needed to make the evidence chain itself verifiable.

a) T6 minimum control bundle

To make T6 verifiable rather than declarative, the framework requires a minimum cross-cutting control record set comprising: — `evidence.checkpoint_id` — `evidence.chain_hash` + `evidence.prev_hash`

— `evidence.signature_ref` + `evidence.hash_alg`
— `retention.policy_id`
— `access.audit_stream_ref`

The cited sources motivate adjacent component primitives relevant to T6: retention-policy guidance [38], tamper-evident log-validation mechanisms [41], and audit-record content guidance [42]. They do not themselves define the full bundle or the exact field set listed above. This study therefore treats the listed bundle as a framework-defined T6 interoperability contract over those separate primitives. A deployment qualifies as T6-conformant only when those controls are emitted and independently verifiable for the investigation window.

R1–R5 and the T6 control bundle define *what* a conformant deployment must guarantee. Those requirements become operational only if heterogeneous records can be normalized into a shared execution model. The next subsection therefore defines the event taxonomy and correlation keys used for that normalization.

B. Event Taxonomy and Correlation Keys

The unit of execution profiling is a session-scoped execution graph rather than an isolated log line; a session timeline is an ordered projection of that graph for analyst interpretation. Seven event types carry the profiling data. E1 anchors the session; E2–E6 capture model I/O, context lineage, tool invocation, identity or token activity, and policy decisions; and E7 records boundary-visible egress.

All event types join through the shared keys {`agent_id`, `session_id`, `trace_id`/`span_id` or `request_id`}. For graph-structured or delegated execution, `parent_span_id`/`parent_request_id`, `tool.call_id`, and `delegation.chain_id` become conditionally REQUIRED.

E7 alone is profile-aware: the network-observable profile supports bounded egress analysis, while the join-baseline profile supports deterministic cross-EP reconstruction when the join conditions hold. This difference between the network-observable and join-baseline E7 profiles traces back to the Section III finding that HTTP/2 multiplexes request semantics within one connection [2], making it impossible to attribute individual requests at Layer 4 alone. QUIC [3] and HTTP/3 [4] add transport-level encryption that further limits boundary-only observation. Deterministic reconstruction therefore depends on propagated L7 correlation keys such as `trace_id` or `request_id`. W3C Trace Context provides a standard correlation mechanism [18].

Throughout this section, EP1–EP5 denote endpoint/runtime evidence, identity/tokens, model/policy, tool path, and egress boundary observation points, respectively.

Figure 1 provides the overview: EP-based streams are normalized into the minimum schema and then joined by shared correlation keys. That processing flow is the formal basis for the schema and event tables below.

Figure 1 and the E1–E7 taxonomy provide the common execution model for the scoped claim below. The claim tests whether these normalized event roles and shared join surfaces are sufficient for FQ1–FQ6 under the stated assumptions, not whether any vendor-specific log format is universally sufficient.

a) Scoped sufficiency claim: This is a scoped engineering claim rather than a universal proof of completeness or minimality. Under the incident classes and evidentiary assumptions of Section II-C and the BP1–BP5 deployment lens, the proposed schema is designed to be sufficient for FQ1–FQ6 when four conditions hold:

1) EP1–EP5 emit E1–E7 with the REQUIRED fields in Table VIII;
2) those records propagate the shared join keys {`agent_id`, `session_id`, `trace_id`/`span_id` or `request_id`} into E6 and join-capable E7 records;
3) the operating conditions in Section IV-B are satisfied; and
4) the join-key rules in Section IV-C3 are followed.

For linear execution paths, session/request identifiers are sufficient for the scoped joins. For graph-structured paths with fan-out, repeated tool invocations, or delegated runtimes, full graph-causal reconstruction additionally requires parent-child lineage anchors and delegated-path identifiers where such execution shapes exist.

If only the E7 network-observable profile is available (without L7 join keys), FQ5, the egress-attribution branch of FQ1, and the egress side-effect branch of FQ2 become partially answerable via heuristic correlation with reduced evidentiary confidence. Policy-effect attribution in FQ1 can still rely on E6 joins, and the core action chain in FQ2 remains answerable from E2/E4 joins. More broadly, when only a subset of the four conditions holds, reconstruction capability degrades non-uniformly across forensic questions; Appendix E characterizes the per-axis consequences.

b) Operational conditions for deterministic joins: As used here, "deterministic" means that the relevant cross-EP joins are resolved by emitted identifiers and comparable event timestamps, rather than by flow/timestamp heuristics alone. The minimum operating conditions are:

- participating systems preserve comparable event timestamps, distinguished from collector-observation time where both exist;
- reconstruction relies on event time and explicit join keys rather than collector arrival order when export paths are buffered or asynchronous;
- logs record identifier issuer/namespace semantics and any boundary translation needed to interpret joins correctly;
- when REQUIRED records are missing, redacted, or otherwise unavailable, that limitation must be detectably signaled rather than overstated (detectable-gap rule).

The detectable-gap rule is motivated by guidance on correlation [18], audit-record content [42], and integrity-failure visibility [19], [41], which together support preserving observability and evidence-integrity primitives at each layer.

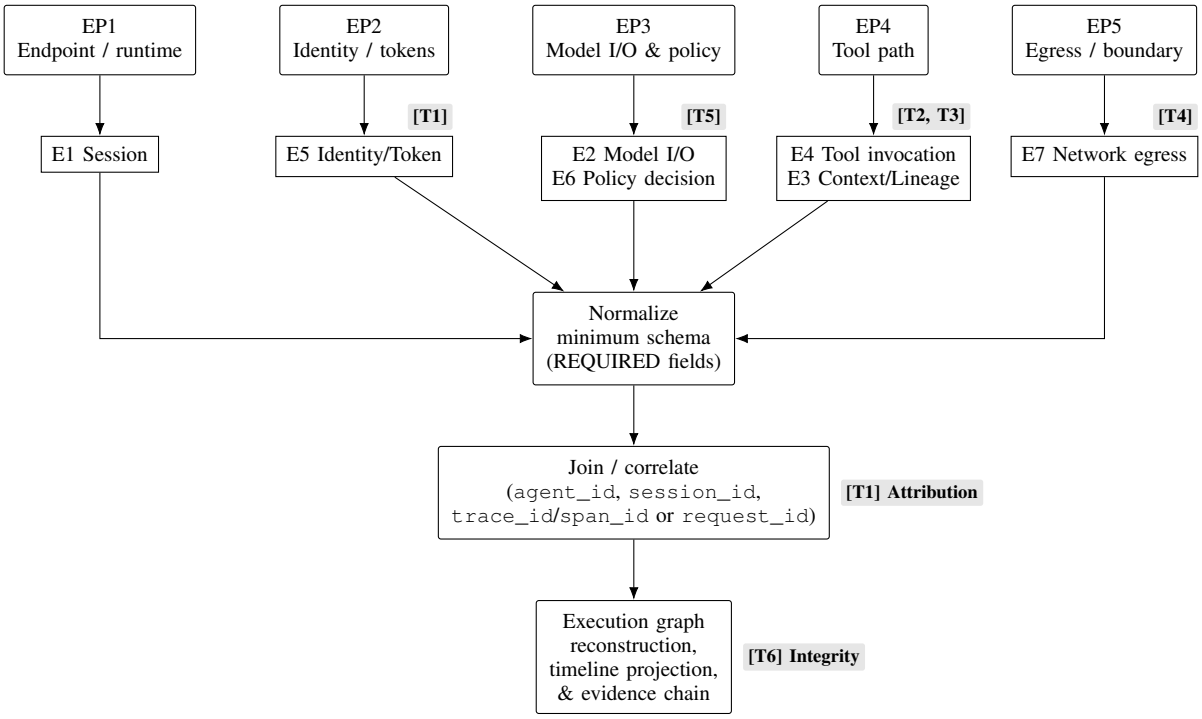

Fig. 1. Unifying view: EP1–EP5 generate heterogeneous event streams (E1–E7) which are normalized to a minimum interoperable schema, joined by correlation keys, and used for execution-graph reconstruction with timeline projection. Overlaid tags [T1]–[T6] show where the Minimum Practical Requirements primarily map to these streams.

The schema subsection below formalizes the corresponding field families and per-event minimum sets. Figure 2, presented later as a synthesis panel, then summarizes how the scoped forensic questions map to axes and REQUIRED event bundles without replacing the normative conformance tables.

C. Minimum Interoperable Schema

To make those joins operational across heterogeneous sources (proxies, MCP gateways, IdPs, provider logs, endpoint tracing), the framework specifies the following minimum interoperable schema as the interoperability contract. Under the operational conditions in Section IV-B, EP1–EP5 records are normalized into E1–E7, joined on shared identifiers, and projected into a session-scoped execution graph. Deterministic reconstruction uses join-capable E7 records; when only the network-observable E7 profile is available, the egress-attribution branches fall back to weaker timestamp/flow alignment. Appendix E provides the corresponding scope-relative failure-mode justification.

Field-selection logic: The schema prioritizes six field families:

- timeline ordering (`timestamp`);
- stable attribution (`agent_id`, `owner`);
- correlation and lineage (`trace_id`, `session_id`, parent-child anchors);
- policy-decision outputs (`decision`, `policy_id`);
- egress quantification (`bytes`, `tokens`); and
- evidentiary properties (integrity, retention, and T6 evidence-control references).

1) REQUIRED vs. OPTIONAL fields: **Recurrence and necessity:** A field receives R when recurrence support is demonstrated either across domain classes or via a standards-backed core primitive that is independently observed in a separate operational artifact class (e.g., OTel spans, gateway logs, or provider audit records). A field receives N when, even without strong recurrence, the scoped substitute test in Appendix E shows that removing it makes at least one mapped forensic question unanswerable or materially weaker. Fields satisfying both tests are marked R+N. Fields that become necessary only under fan-out, repeated child actions, or delegated execution are marked C and activate only under those execution shapes.

Conformance levels: We distinguish between *schema conformance* and *reconstruction-ready conformance*. The first asks whether the REQUIRED fields in Table VIII are emitted at all. The second asks whether those fields can actually support deterministic reconstruction; in practice, that means comparable timestamps across EPs, logged identifier translations, and detectable evidence gaps. A deployment may therefore satisfy the schema and still fall short of reconstruction-ready use. Conformance also requires that REQUIRED joins be satisfied without persisting prohibited secrets in raw form. Records containing unredacted bearer tokens, session cookies, API keys, or full authorization headers are treated as non-conformant under R5. For FQ6, claims about integrity, retention, and evidence control are valid only when the T6 minimum control bundle is present and verifiable.

E7 profile-aware conformance: For E7 specifically, con-

formance is profile-aware. The *network-observable profile* requires only fields that can be emitted from EP5 without L7 interception. The *join-baseline profile* adds attribution and request-level join keys so that E7 can participate in deterministic cross-EP reconstruction when the operational preconditions hold.

How to read Tables VII–VIII: Table VII is a field-family dictionary. Table VIII is the normative per-event checklist. E1 provides the anchor keys, E2–E6 inherit E1 and add event-specific fields, and E7 is the only profile-aware event type. Conditional REQUIRED fields activate only for graph-structured or delegated execution paths.

Appendix evidence trail: Appendix A provides the auditable field-family evidence trail for Tables VII and VIII, including recurrence support and REQUIRED basis. The main text states only the normative promotion and conformance rules.

Note: For the schema summaries in Tables VII and A2 (the latter in Appendix A), canonical field-family alignment is used for recurrence accounting and comparison only. Normalized records MUST preserve source-scope qualifiers.

2) Event-type-specific minimum sets (E1–E7): Because different observation points emit different event shapes, we derive per-event minimum sets from T1–T6 and the core schema in Table VII. Table VIII specifies REQUIRED and OPTIONAL fields per event type; the case study later uses these sets as a comparison baseline when discussing visible signals and missing evidence classes in the reviewed direct-evidence corpus. Conditional REQUIRED fields apply only when the execution shape includes child operations, repeated tool calls under a shared session/trace, or delegated execution hops.

T6 cross-cutting note: T6 is cross-cutting and REQUIRED at pipeline/storage scope. Minimum control-bundle fields are `evidence.checkpoint_id`, `evidence.chain_hash`, `evidence.prev_hash`, `evidence.signature_ref`, `evidence.hash_alg`, `retention.policy_id`, and `access.audit_stream_ref`; see the T6 minimum control bundle paragraph and Listing 5. Fields labeled conditional REQUIRED in the REQUIRED column are mandatory only when the execution path satisfies the stated condition; otherwise they are not required for baseline conformance.

E6 baseline semantics: Baseline E6 conformance requires a policy-decision record that evidences the evaluated policy and decision (`policy_id`, `decision`), a decision instance handle (`decision_id`) for joins, comparable event time, and the shared join anchor ((`trace_id`+`span_id`) or `request_id`) needed to bind the decision to the governed action.

The cited sources illustrate product-specific policy or approval surfaces: Model Armor [61], Prompt Shields [62], Bedrock Guardrails [63], and OPA decision logs [52]. If a deployment inserts a human approval workflow, `approval.ref`, `approval.actor`, and `approval.phase` MUST be emitted as a correlated approval-extension record joinable to the governed action (a deployment-level obligation). For this study's recurrence accounting, however, those fields are excluded from the baseline E6 REQUIRED family count because the surveyed corpus does not establish them as a universal recurrent field set. That exclusion is this study's normalization choice, so approval-workflow fields remain deployment-specific extension fields rather than surveyed baseline fields.

Figure 2 is a synthesis panel rather than a second conformance definition. It summarizes how the scoped forensic questions map to axes and REQUIRED event bundles after Tables VII and VIII have fixed the normative baseline.

3) Join-key generation and propagation rules: The join-key rules fall into four groups: (1) core join keys, (2) graph and delegation keys, (3) propagation and fallback, and (4) join hygiene.

1) Core join keys:

The grouped bundle below is a framework-defined interoperability set; citations remain attached to the individual primitives rather than to the bundle as a pre-existing standard.

(i) `agent_id`: NHI identifiers should be bound to executing-environment identity evidence where supported; SPIFFE/SPIRE provide relevant workload-identity primitives [9], [10], [11]. When an enterprise-issued stable workload identity is not available (common in BP5), `agent_id` should be explicitly marked as vendor-scoped, and joinability requirements shift to the vendor's exposed logs and contractual telemetry.

(ii) `session_id`: `session_id` is created at session start (E1) and must be propagated through model, tool, and gateway layers. If the architecture includes multiple tool executors or delegated runtimes, the session identifier must remain stable across those hops.

(iii) `trace_id/span_id`: `trace_id`/`span_id` follows W3C Trace Context where feasible [18]. `request_id` fallback is a framework-defined interoperability aid at boundaries that do not support tracing.

2) Graph and delegation keys:

(iv) Parent-child lineage: Trace Context materials ground interoperable trace linkage [18]. GenAI span conventions provide adjacent span-hierarchy examples within instrumented GenAI traces [59]. Within this framework, when such linkage is emitted, logs MUST preserve an equivalent parent-child anchor across model, tool, policy, and join-capable egress events. When only `request_id` is available, linear joins remain possible, but parent-child causality across parallel branches is only partially reconstructible unless an equivalent parent-child link is emitted. The broader cross-event requirement and the deterministic-causality distinction are framework-defined.

(v) Invocation identity: MCP tool-path materials document the tool boundary and adjacent invocation surfaces [23]. GenAI span conventions more directly support stable per-invocation handles [59]. Within this framework, parallel or repeated tool actions under a shared session/trace SHOULD emit

TABLE VII
CORE FIELDS OF THE MINIMUM INTEROPERABLE SCHEMA (SUMMARY). NOTE: THIS TABLE IS A COMPACT SUMMARY REFERENCE; NORMATIVE REQUIRED/OPTIONAL SETS ARE DEFINED PER EVENT TYPE IN TABLE VIII.

Minimum field	Meaning	Standard alignment (example)	Maps to
`timestamp`	Timeline ordering.	OTel LogRecord `time_unix_nano` [19].	Common to T1–T6
`agent_id`, `owner`	Execution-actor attribution (with scope preserved).	`agent_id`: SPIFFE/SPIRE workload-identity primitives where available [9], [10], [11]. When only application-scoped identifiers are exposed, implementations may emit application-level identifiers such as `gen_ai.agent.id` [39]. This framework can retain those values in a framework-normalized application-level identifier family while preserving emitted scope explicitly. `owner`: OIDC `iss`+`sub` provides subject/issuer claims that can contribute to owner attribution [50]. OTel logs/resources provide a generic carrier for exporting such identifiers [19]; they are not a co-equal semantics source. The retained pair is a framework-qualified attribution bundle rather than a universal source-native paired schema.	T1
`session_id`	Qualified session anchor.	`session.id` denotes client/application-session semantics [48]. `gen_ai.conversation.id` denotes conversation/thread semantics where exposed [39]. This framework treats the retained field as a qualified session anchor, preserves the emitted scope explicitly, and does not treat it as a universal literal session key.	T1
`trace_id`, `span_id` (or `request_id`)	Request/span correlation.	W3C `traceparent` + OTel correlation [18], [19].	T1
`parent_span_id` or `parent_request_id`	Parent-child causality anchor for graph-structured execution.	Trace parent linkage [18] directly supports `parent_span_id`-style trace linkage. GenAI span conventions [59] provide adjacent span-hierarchy context within instrumented GenAI traces. The retained `parent_span_id`/`parent_request_id` family, including the `parent_request_id` fallback, is a framework-defined graph-lineage normalization rather than a source-native literal field.	T2 (graph causality)
`tool.call_id`	Stable invocation identity for parallel or repeated tool actions.	GenAI/tool span conventions and tool-call identity [59].	T2 (graph causality)
`delegation.chain_id`	Delegated execution lineage across token exchange / subagent / multi-hop paths.	RFC 8693 token-exchange semantics [60] directly support delegated-path token-exchange relationships. RFC 9700 security guidance [33] provides adjacent delegated-token security context. The retained `delegation.chain_id` family is a framework-defined lineage normalization rather than a source-native literal field.	T1/T5 (delegation)
`process.pid` (or equivalent process anchor)	Runtime process anchor for host-level flow correlation.	Implementation-specific process anchors may be carried in telemetry records where available; OpenTelemetry Logs provides a generic carrier model [19].	T1 (support)
`dst.addr`, `dst.port`	External destination IP/port (for joining flows and NAT translation logs).	Canonical destination-field family; see Appendix A for mappings to `network.peer.*` (preferred) and `net.sock.peer.*` (deprecated).	T4 (correlation)
`gen_ai.operation.name`, `tool.name`	Model/tool event type.	GenAI attributes and span conventions [39], [59].	T2
`policy_id` + `decision`	Policy-decision outcome (e.g., allow/block/transform). If a deployment inserts a separate human approval step, approval state is represented by a deployment-specific approval extension rather than overloaded into the baseline decision field.	XACML policy/decision semantics [51]. OPA decision-log schema examples and join handles [52]. The retained `policy_id` + `decision` pairing is framework-normalized rather than a universal source-native field set.	T5
`egress_dest` + `bytes`/`tokens`	External destination/exposure volume.	Cloudflare AI Gateway DLP feature docs support boundary scanning [29]. Request logging docs support exposed request metadata [65]. `network.peer.address` / `network.peer.port` ground destination fields [53]. Byte-volume elements such as IPFIX octet counters ground byte-volume [55]. Provider token-count metrics support token counts where documented [64]. The combined egress field family remains framework-normalized rather than a single source-native common schema.	T4

TABLE VIII
EVENT-TYPE-SPECIFIC MINIMUM FIELD SETS (REQUIRED VS. OPTIONAL) FOR E1–E7.

Event type	REQUIRED minimum fields	OPTIONAL (recommended) / Conditionally REQUIRED when a separate approval workflow exists
E1 Session	`timestamp`, `event_type`, `agent_id`, `owner`, `session_id`	`host.id`, `process.pid`, `runtime.name/version`, `workspace.id`, `deployment.bp`
E2 Model I/O	E1 keys + `trace_id`+`span_id` (or `request_id`), `gen_ai.operation.name`, `model`, `direction` (in/out), `parent_span_id` or `parent_request_id` (conditionally REQUIRED for child operations in graph-structured paths)	`prompt_digest`, `response_digest`, `digest.alg`, `digest.key_ref`, `redaction.applied`, `redaction.rule_id`, `secret.findings_count`, `token_counts.*`, `content_labels.*`
E3 Context/Lineage	E1 keys + `trace_id`+`span_id` (or `request_id`), `context.source_id`, `context.type`, `context.ref` (URI/path), `context.digest`	`context.sensitivity`, `retrieval.query_digest`, `tool_descriptor.digest`
E4 Tool invocation	E1 keys + `trace_id`+`span_id` (or `request_id`), `tool.name`, `tool.server_id`, `tool.action` (call/list), `outcome`, `tool.call_id` and `parent_span_id` or `parent_request_id` (conditionally REQUIRED for repeated or sibling child tool actions in graph-structured paths)	`tool.input_digest`, `tool.output_digest`, `digest.alg`, `digest.key_ref`, `redaction.applied`, `redaction.rule_id`, `secret.findings_count`, `mcp.session_id`, `policy_decision_id`
E5 Identity/Token	E1 keys + `identity.subject`, `issuer`, `token.event` (issue/refresh/revoke), `token.audience`, `delegation.chain_id` (conditionally REQUIRED for on-behalf-of, token-exchange, subagent, or multi-hop delegation paths)	`scope`, `expires_at`
E6 Policy decision (+ approval extension where separately instrumented)	E1 keys + `trace_id`+`span_id` (or `request_id`), `policy_id`, `decision`, `decision_id`, `delegation.chain_id` (conditionally REQUIRED for delegated execution paths), `parent_span_id` or `parent_request_id` (conditionally REQUIRED when the decision governs a child execution node)	`rationale_ref`, `policy.version`, `approval.ref`, `approval.actor`, `approval.phase`
E7 Network egress (profile-aware)	Base REQUIRED (network-observable): `timestamp`, `event_type`, `dst.addr`, `dst.port`, `bytes_out`; Join REQUIRED (deterministic cross-EP reconstruction): Base REQUIRED + `agent_id`, `owner`, `session_id`, `trace_id`+`span_id` (or `request_id`), `parent_span_id` or `parent_request_id` (conditionally REQUIRED for graph-structured paths that need deterministic effect attribution)	`dst.fqdn` (if observable), `tls.sni` (if available), `ja3/ja4` (if used), `nat.translation_id`, `tokens_estimate`

Scoped forensic question	Required axes	Minimum events + REQUIRED fields (Table VIII shorthand)	Failure-mode pointer
FQ1 Accountable attribution	T1 (+T6 trust)	Deterministic mode: E1+E5 joined with E6 and join-capable E7 effects; REQUIRED: E1 keys + (`trace_id`+`span_id` or `request_id`) + {`policy_id`, `decision`, `decision_id`} + E7 join-capable REQUIRED set. If the deployment inserts a human approval step, add correlated approval-extension fields {`approval.ref`, `approval.actor`, `approval.phase`}. Fallback mode: if only E7 network-observable profile is available, policy-effect attribution remains deterministic via E6, while egress-effect attribution is heuristic via EP4/EP5 timestamp-flow alignment.	Remove T1 (T6 trust note)
FQ2 Causal chain reconstruction	T2+T1 (+T4 side effects)	Deterministic mode: E4 actions are first organized as nodes in a session-scoped execution graph and then aligned with E2 and join-capable E7 side effects; REQUIRED: E1 keys + (`trace_id`+`span_id` or `request_id`) + {`gen_ai.operation.name`, `model`, `direction`} + {`tool.name`, `tool.server_id`, `tool.action`, `outcome`} + conditional graph-causality anchors {`parent_span_id` or `parent_request_id`, `tool.call_id`} when actions fan out or repeat + E7 join-capable REQUIRED set. Fallback mode: core E2/E4 action-chain reconstruction remains deterministic, while egress side-effect linkage uses E7 network-observable profile (`timestamp`, `event_type`, `dst.addr`, `dst.port`, `bytes_out`) with heuristic EP4/EP5 alignment.	Remove T2 / Remove T4
FQ3 Data lineage (+ available sensitivity context)	T3+T1	E3 lineage joined to E2/E4; REQUIRED: E1 keys + (`trace_id`+`span_id` or `request_id`) + {`context.source_id`, `context.type`, `context.ref`, `context.digest`}. Where a deployment classifies sensitivity, `context.sensitivity` qualifies that lineage further.	Remove T3
FQ4 Policy/approval path evidence	T5+T1	Baseline E6 policy-decision records are joined to E2/E4/E7 by (`trace_id`+`span_id` or `request_id`) and ordered by timestamp; REQUIRED: E1 keys + (`trace_id`+`span_id` or `request_id`) + {`policy_id`, `decision`, `decision_id`}. If a human approval workflow exists, correlated approval-extension fields {`approval.ref`, `approval.actor`, `approval.phase`} are additionally required to prove the approval path.	Remove T5
FQ5 Exposure accounting & blast radius	T4+T1 (+T2 causality)	Deterministic mode: E7 join-capable profile linked to the relevant E4 execution-graph node; REQUIRED: {`timestamp`, `event_type`, `dst.addr`, `dst.port`, `bytes_out`, `agent_id`, `owner`, `session_id`} + (`trace_id`+`span_id` or `request_id`) + conditional parent-child lineage when multiple child actions share a session/trace. Fallback mode: E7 network-observable profile (`timestamp`, `event_type`, `dst.addr`, `dst.port`, `bytes_out`) with heuristic EP4/EP5 alignment.	Remove T4
FQ6 Evidence reliability	T6	Cross-cutting over E1–E7; REQUIRED: integrity/retention/access-audit controls and redaction evidence at storage/system and export layers (Section IV-A, Appendix E).	Remove T6

Fig. 2. Explicit mapping from scoped forensic questions (FQ1–FQ6) to minimum axes (T1–T6) and REQUIRED field sets (Table VIII), presented as a synthesis panel with corresponding failure-mode results (Appendix E, Table E1). Here, "deterministic mode" denotes graph-aware deterministic joins under the operational preconditions stated in Section IV-B and in Section IV-C3.

a stable `tool.call_id`. That evidence supports a conditionally REQUIRED `tool.call_id` family when downstream effects, approval-extension records, or delegated actions must be attributed to one child tool action among several siblings, rather than establishing a universal source-native field.

(vi) Delegation lineage: Token-exchange semantics directly support delegated-path token-exchange relationships [60]. OAuth security guidance [33] and MCP authorization materials [21] provide adjacent security and authorization context for those delegated paths. Within this framework, those underlying materials are normalized into a stable `delegation.chain_id` or equivalent lineage identifier that is conditionally REQUIRED across E5, E4, E6, and join-capable E7 records, consistent with the conditional REQUIRED status in Table VIII. The cross-hop lineage rule itself is framework-defined.

3) Propagation and fallback:

(vii) Gateway propagation: MCP security guidance documents proxy-boundary constraints [20]. MCP authorization materials document HTTP transport authorization surfaces [21]. Gateway observability materials document logging surfaces [22]. Within this framework, gateways and MCP proxies MUST propagate the selected join keys into tool invocations, policy-decision events, and any approval-extension events. When identifier translation occurs, they SHOULD log both inbound and outbound identifiers. The propagation rule itself is framework-defined.

(viii) Fallback mechanisms: In delegated SaaS or external inference environments (e.g., BP4/BP5), L7 correlation identifiers may be unavailable or not propagated across boundaries, breaking deterministic cross-system joins. The architecture may then fall back to heuristic correlation (e.g., aligning EP5 egress timestamps and 5-tuple/NAT metadata with EP4 gateway logs). This fallback corresponds to the E7 network-observable profile in Table VIII. Where applicable, minimum interoperable schemas and correlation-key propagation should be treated as contractual telemetry requirements.

4) Join hygiene:

(ix) Uniqueness and namespace: If `request_id` is used as a fallback join key, it MUST be collision-resistant at least within (`owner`, `agent_id`, `session_id`, bounded time window). When identifiers are emitted by multiple systems (e.g., gateways and SaaS vendors), logs MUST record the issuer or apply an explicit namespace to prevent false joins across concurrent sessions or tenants. If an identifier is translated at a boundary, inbound and outbound identifiers MUST be linked in the same evidence graph.

(x) Event-time ordering: Deterministic cross-EP reconstruction requires comparable event timestamps across participating systems. OpenTelemetry log records provide timestamp fields and can preserve both event time and collector-observation time when a telemetry pipeline emits them [19]. In this framework, event time is used as the primary ordering key and observation time is retained to diagnose buffering or

export delay; that ordering rule is framework-defined rather than source-native.

(xi) Gap signaling: Integrity-failure visibility and audit-control sources provide adjacent integrity and audit primitives [41], [42]. In this framework, if REQUIRED records are dropped, redacted beyond joinability, or otherwise unavailable for an investigation window, the telemetry pipeline SHOULD preserve detectable evidence of that limitation (e.g., chain-break, validation failure, or explicit collection-gap signaling) so affected forensic questions can be downgraded explicitly in evidentiary confidence rather than overstated. The detectable-gap and downgrade rule itself remains framework-defined.

4) Illustrative JSON examples: Appendix C provides sanitized direct-evidence raw-anchor excerpts and a supplemental ancillary surface table. It also retains minimal JSON examples for a tool-call event (E4), a policy-decision event (E6), and a network egress event (E7) as schema illustrations.

D. Implementation Paths (Technology-Agnostic)

Because the Section III evidence does not support a single mandatory implementation stack, this framework specifies implementation paths by the minimum signals they can contribute. Each path can satisfy only a subset of the required fields rather than providing a standalone route to full conformance; in practice, most deployments must combine several of them while propagating the shared join keys across EPs:

- **Application/SDK tracing:** can contribute generic event and log carriers [19]. GenAI conventions provide model-I/O fields [39], while GenAI span conventions provide tool-call-specific fields [59]. GenAI metrics conventions offer complementary quantitative telemetry [40]. Policy-event fields prescribed by this framework (T5) can be attached to those spans when instrumented.
- **MCP gateway/proxy:** MCP security and authorization materials motivate tool-path security and authorization primitives [20], [21], while gateway observability docs ground request logging [22]. In this framework, that implementation path can act as a centralized mediation layer, but that centralization is the framework's implementation reading rather than a source-native conformance route.
- **Guardrails and DLP at model/egress boundaries:** can produce policy-assessment outputs for screened model I/O [61], [62], [63]. Where the product documents it, they can also expose token counts [64]. Some products separately expose per-request DLP actions [65]. Cloudflare's DLP feature page [29] supports the boundary-scanning capability itself. The cross-product T4/T5 grouping used here remains framework-defined rather than a single source-native schema.
- **Identity/token layer:** Workload-identity and token-management materials motivate adjacent identity and token-management primitives [9], [10], [11], [33]. In this framework, that layer should record issuance, refresh, and usage events for attribution and misuse analysis (T1). Those records are normalized here as a paper-defined E5 identity/token event family rather than a universal source-native event schema.
- **Evidentiary controls (T6):** retention-policy controls [38], tamper-evident validation [41], and audit-control references [42] motivate adjacent T6 component primitives. The full T6 evidence-control bundle remains framework-defined, and access-audit references are treated as part of that framework-defined bundle rather than as universally source-derived fields.

These implementation paths matter analytically because they determine which schema fields and join conditions can be evaluated directly and which can only be assessed under narrower artifact-based visibility postures.

E. Evaluation Plan

Having specified the minimum schema and the implementation paths that can contribute to it, the remaining step is to define how schema conformance and reconstruction readiness are evaluated. The framework defines four evaluation dimensions (M1–M4) organized into two tiers that reflect different experimental prerequisites. Tier 1 has two evidentiary postures: with join-complete raw data, it supports direct M1/M2 scoring; with an intercepted artifact corpus, it supports only a visibility baseline for applicable field families and join surfaces. Section V uses only the latter posture. Tier 2 targets quantitative accuracy and cost measurement from paired experiments. Both tiers are specified here so that the evaluation design is self-contained and future replication is precise.

1) Tier 1: visibility assessment (M1, M2): Tier 1 dimensions assess whether the minimum schema surfaces sufficient evidence for cross-EP reconstruction. With join-complete raw data, they support direct assessment of field coverage and deterministic join success. With an artifact-driven intercepted corpus, they support only a visibility-baseline reading of applicable field families and join surfaces.

M1 (field coverage): proportion of applicable Table VIII REQUIRED field families that are directly evidenced in the evaluated corpus.

M2 (correlation success): proportion of scoped cross-EP joins required by FQ1–FQ6 that are satisfied deterministically by shared identifiers and comparable event timestamps, without heuristic fallback.

2) Tier 2: paired empirical validation (M3, M4): Tier 2 dimensions require paired baseline-versus-instrumented experiments with request-level transfer ground truth—an experimental setup beyond the scope of the current artifact-based study. They are retained here to complete the evaluation design; their instantiation is deferred to future work.

M3 (egress quantification error): deviation between telemetry-derived exposure quantities and request-level ground truth in paired baseline-versus-instrumented runs.

M4 (overhead): additional latency, compute cost, and storage or logging volume introduced by instrumentation relative to baseline.

TABLE IX
DIRECT-EVIDENCE REVIEW CENSUS AND REPRESENTATIVE ANCHORS REPRODUCED IN SECTION V.

Recorded run	Total wrappers	Rows reviewed	Anchors in text	Census only
`Cursor(Auto) Test1`	3550	3550	5	3545
`Cursor(Auto) Test2`	2607	2607	2	2605
`GPT 5.4 IDE Ext. Test1`	874	874	2	872
`GPT(Codex 5.3) IDE Ext. Test1`	1130	1130	3	1127

3) Case-study measurement posture: Because the case study is artifact-based, it does not include full raw event logs or full network-flow ground truth. Accordingly, Section V reports only the Tier 1 visibility-baseline posture: a transparently bounded direct-evidence corpus with representative endpoint-family observations and explicit non-claims, rather than direct empirical M1/M2 conformance scores.

Here, transparency refers to explicit coding rules, fixed evidence boundaries, and representative sanitized excerpts. It does not imply public release of the full intercepted corpus.

Section IV defines the MPR framework and the conditions for reconstruction-ready cross-EP joins. Section V then uses that framework only as a scoped Tier 1 visibility baseline for an IDE-based agent proxy corpus, asking which axis-relevant signals are directly visible and which evidence classes remain partial or absent.

V. CASE STUDY: VISIBILITY ASSESSMENT IN AN IDE-BASED AGENT PROXY CORPUS

A. Corpus Scope and Evidence Boundary

Consistent with that scoped evaluation posture, this case study evaluates four intercepted runs (8,161 raw wrappers) at a Tier 1 direct-evidence visibility baseline. Direct claim support is restricted to raw request-response wrappers in the reviewed `recorded-*` directories; extracted or decoded derivatives and user-authored artifacts remain support-only. Because the proxy path is incomplete, proxy silence is treated as a capture limit rather than as evidence of behavioral absence. Table IX summarizes the reviewed direct-evidence census that bounds the claims that follow in this section.

Note: All prevalence statements in Section V derive from the full reviewed direct-evidence census. The section reprints only a small anchor subset for auditability and readability; the remaining wrappers stay census-only.

The reviewed corpus differs not only by endpoint class (Cursor RPC vs. GPT REST vs. MCP) but also by raw host-family balance. Table X shows that balance before the subsection-level analysis begins.

At this direct-evidence baseline, the reviewed raw corpus does not directly show first-class policy-decision records, approval-workflow evidence, or an evidence-integrity control bundle. Table XI summarizes this visibility picture across T1–T6.

Here T1–T6 retain the Section IV meanings of attribution, tool-path activity, context lineage, egress exposure accounting, policy or approval path, and integrity, retention, or access audit.

Table XI shows that the current corpus is strongest on setup/control and bounded endpoint/channel visibility, while T3 is not directly evidenced and T5/T6 currently lack direct support in the reviewed raw corpus. These raw findings answer only the first part of RQ3: which surfaces are directly visible in the proxy path. The analysis proceeds in three layers: first establishing what the raw corpus directly shows (B), then assessing what derived material can recover and how the recovered layers relate (C–F), and finally testing those findings through cross-run and cross-ecosystem comparisons (G–I) before synthesizing residual gaps and explicit non-claims (J–K; Table XX).

B. Repeated Cursor Control-Plane RPC Traffic

The two recorded Cursor(Auto) runs both include `POST /aiserver.v1.BidiService/BidiAppend` to `api2.cursor.sh` with HTTP status `200`. Across the complete direct-evidence review, BidiAppend accounts for 2,155 of 3,550 raw rows in Cursor(Auto) Test1 and 1,866 of 2,607 raw rows in Cursor(Auto) Test2. Table XII summarizes that concentration using the full review census rather than an isolated example trace.

Note: concentration demonstrates recurrence, not full action semantics.

The census confirms recurrence and dominance for one Cursor control-plane RPC family, relevant to the transport surface where T2 (tool-invocation causality) signals would appear. It does not establish E4 conformance, because the raw wrappers lack the E4 minimum set (`tool.name`, `tool.server_id`, `tool.action`, `outcome`, stable invocation identity).

The raw tier establishes transport-level recurrence but not action-level semantics. The next subsection asks what semantic classes BidiAppend derivatives recover, and the following subsection asks whether RunSSE preserves a stronger session-level view. In both cases, derived material remains support-only and does not upgrade absent direct evidence.

C. Support-Derived Fragment Recovery in BidiAppend

Derived Cursor materials provide a second, support-only view of the same sessions whose raw tier is dominated by framed BidiAppend traffic. Table XIII summarizes the derivative footing before the BidiAppend and RunSSE layers are assessed separately.

Table XIV organizes the strongest recovered semantic families and structural character visible across the four Cursor derivative sets, while preserving the distinct recovery ceiling of each layer.

BidiAppend decoded derivatives enable incomplete fragment-level semantic recovery across the two Cursor(Auto) runs: readable fragments are limited to a minority of decoded files (Table XIII), but the recovered classes—recurring prompt, skill, plan, settings, and code-related families—are

TABLE X
COUNT-RANKED RAW HOST-FAMILY MIX ACROSS THE FOUR REVIEWED RUNS IN THE DIRECT-EVIDENCE CORPUS.

Recorded run	Top raw host families (count-ranked)	Reading
Cursor(Auto) Test1	api2.cursor.sh 3171; chat.openai.com 101; mcp.notion.com 91; files.pythonhosted.org 78; pypi.org 39	The raw mix is overwhelmingly api2.cursor.sh driven, with smaller OpenAI, Notion MCP, and package-retrieval families.
Cursor(Auto) Test2	api2.cursor.sh 2491; mcp.notion.com 32; mobile.events.data.microsoft.com 25; api3.cursor.sh 20; files.pythonhosted.org 10	The same dominant control-plane host remains, but the host mix is narrower and more concentrated than in Test1.
GPT 5.4 IDE Ext. Test1	api2.cursor.sh 470; chat.openai.com 150; mcp.notion.com 144; ab.chatgpt.com 39; chatgpt.com 21	The raw mix combines shared Cursor/OpenAI and MCP surfaces with additional experimentation and ChatGPT control-plane families.
GPT(Codex 5.3) IDE Ext. Test1	api2.cursor.sh 434; chat.openai.com 213; mcp.notion.com 210; chatgpt.com 167; ab.chatgpt.com 43	The GPT-side mix is broader, with substantial chatgpt.com response-channel traffic added to the shared families.

TABLE XI
VISIBILITY ASSESSMENT AGAINST THE SECTION IV BASELINE USING THE REVIEWED DIRECT-EVIDENCE CORPUS.

Axis	Closest baseline in Table VIII	Visibility	Representative anchors	Safe interpretation
T1	E1 / E5 / join-capable E7	Partial	mcp-session-id header; ACCOUNT-01-class usage fields	The corpus shows an MCP session header and usage-account metadata, but it does not show stable actor binding or request-level cross-source joins.
T2	E4	Partial	BidiAppend recurrence; MCP initialize	Recurring control-plane and setup traffic are visible, but direct E4 minimum-set conformance is not shown.
T3	E3	Not directly evidenced	None in the current direct anchors	No direct context-source or provenance evidence is shown in the current raw corpus.
T4	E7	Partial	/backend-api/codex/responses; /backend-api/wham/usage; telemetry/metrics endpoints	Bounded endpoint/channel visibility only; no deterministic request-to-effect attribution is shown.
T5	E6 (+ approval extension)	Absent in current direct corpus	None	No policy-decision or approval-workflow evidence claim is supported by the current direct-evidence corpus.
T6	Cross-cutting	Absent in current direct corpus	None	No evidence-control claim is supported by the current direct-evidence corpus.

analytically identifiable even though the outputs remain mixed, overlapping, and non-transcript-like rather than document-complete payloads.

Within that incomplete recovery, the two Cursor runs expose different BidiAppend recovery regimes. In Test1, the readable material behaves like sliding content windows over a mixed stream, with repeated skill and settings tails recurring across 138 decoded files. In Test2, 1,866 raw shards collapse into just two decoded aggregates. Despite that asymmetry, both runs preserve recurring fragment classes (Table XIV). This remains an interpretive reading and does not turn BidiAppend into direct action-level evidence.

TABLE XII
BIDIAPPEND CONCENTRATION WITHIN THE TWO RECORDED CURSOR(AUTO) RUNS.

Recorded run	BidiAppend	Total	Share
Cursor(Auto) Test1	2155	3550	60.7%
Cursor(Auto) Test2	1866	2607	71.6%

D. Support-Derived Session-Level Recovery in RunSSE

RunSSE decoded derivatives provide a stronger but still incomplete recovery layer. Across the same two Cursor(Auto) runs, they support substantial but incomplete session-level visibility: planning, implementation, validation, reporting, and verification phases become visible, but the stream remains mixed, noisy, and non-equivalent to a direct transcript.

In Cursor(Auto) Test1, the first decoded file preserves the system prompt and user workload most clearly, while later files retain backend file-URL anchors, plan references, implementation updates, validation steps, and report-writing requests. In the derived support material, the heaviest recovered session body reaches 45,606 frames and 1,543 text blocks, including 124 assistant and 70 tool messages. One mid-run file alone concentrates 23 AskQuestion references.

In Cursor(Auto) Test2, the first decoded file behaves like planning, task definition, and architecture setup, whereas the second concentrates most of the execution, fix, and verification material. The preserved architecture overview and Mermaid-form design material stay concentrated in the first stage, while the second reaches 78,759 frames with 184 assistant, 170 tool, and 92 reasoning entries. Even here, however, the only direct evidence in this case is still the

TABLE XIII
DECODE COVERAGE FOR CURSOR(AUTO) BIDIAPPEND AND RUNSSE DERIVATIVES, USED ONLY AS SUPPORT MATERIAL.

Recorded run	Raw files	Extracted BidiAppend	Decoded BidiAppend	Extracted RunSSE	Decoded RunSSE	Reading
`Cursor(Auto) Test1`	3550	2155	138 files; 11 `OK`, 127 `GZIP_FAIL;ZLIB_FAIL`	11	11 files; text-block summaries from 78 to 1543 blocks	BidiAppend fragments recover prompt, skill, plan, settings, and code-related classes; RunSSE yields stronger session-level recovery, but neither layer is transcript-complete.
`Cursor(Auto) Test2`	2607	1866	2 files; both `GZIP_FAIL;ZLIB_FAIL`	2	2 files; both `OK`	BidiAppend fragments recover netprof- and architecture-related classes; RunSSE yields a clearer two-stage session flow, but both layers remain incomplete.

TABLE XIV
RECOVERED SEMANTIC FAMILIES ACROSS THE FOUR CURSOR(AUTO) DERIVATIVE SETS, USED ONLY AS SUPPORT MATERIAL.

Derivative set	Recovered classes	Structural character	Analytical use
`Cursor(Auto) Test1` BidiAppend	Workload prompt, recurring skill-document families for skill creation or installation, PDF or document handling, settings update, description-like instruction blocks, settings-guide text, mini-queue plan-path references, Python code fragments, and local path classes.	Sliding-window recovery with recurring tails and mixed document strata.	Supports substantial but incomplete fragment-level recovery without implying document-complete payloads.
`Cursor(Auto) Test2` BidiAppend	Offline netprof workload prompt, skill-creation metadata, repeated settings-guide tails, and mixed path traces.	1,866 shards collapse into two decoded aggregates, 1,723 under `8KB`, and only two at or above `500KB`.	Supports the same fragment-level verdict under a thinner and more asymmetric recovery regime.
`Cursor(Auto) Test1` RunSSE	System prompt, user workload, backend file-URL anchors, plan references, implementation updates, validation steps, report-writing requests, and tool-turn concentration.	Session-like recovery with a heavy body at 45,606 frames and 1,543 text blocks.	Supports a stronger session-level recovery verdict than BidiAppend while remaining support-only and non-transcript-complete.
`Cursor(Auto) Test2` RunSSE	System prompt, plan frontmatter, architecture overview, Mermaid-form design material, implementation or fix traces, and verification flow.	Strong two-stage split between planning or architecture setup and the main execution body at 78,759 frames.	Supports the same session-level verdict with a clearer two-stage split than the mini-Celery run.

raw RunSSE wrappers, which expose only transport-framed `text/event-stream` plus `base64:` content.

E. Cross-Layer Relationship Between BidiAppend and RunSSE

Across both Cursor runs, the raw wrappers directly show recurring BidiAppend and RunSSE transport families, but they ground only transport coexistence, not shared session semantics. Table XV summarizes support-guided alignment cues over matched request and session identifiers. Those support-guided cues are consistent with a two-level interpretation in which BidiAppend preserves request-fragment views and RunSSE preserves a richer event-stream view.

Across both runs (Table XV), the reviewed support material is consistent with field 2 acting as the request identifier (exact agreement across all raw files) and field 3 acting as a request-scoped append sequence that resets at request boundaries. The RunSSE request sets match exactly, and both runs show one-session alignment in the reviewed support material. This reading is support-guided; it does not establish E4 minimum-set conformance, direct tool-action reconstruction, or a complete execution graph.

Aggregate statistics support the same bounded reading: BidiAppend is the top-count family in both runs, while RunSSE appears less often but offers richer session-level cues in the decoded support material.

F. MCP Setup and Discovery

Table XVI turns the `initialize` exchange into a worked setup trace.

Raw setup and discovery traces show that MCP-related setup surfaces appear in both Cursor and GPT-side runs, which provides a narrow basis for later cross-ecosystem comparison. Cursor(Auto) Test1 records `POST /mcp` to `mcp.notion.com` with JSON-RPC method `initialize`, HTTP status `200`, and `text/event-stream` response content. `GPT(Codex 5.3) IDE Ext. Test1` separately records `GET /.well-known/oauth-authorization-server` to the same host with HTTP status `200`.

Together these raw rows show that MCP-related setup and discovery traffic is observable across at least two runs in the reviewed corpus. These remain setup-only: relevant to partial tool-path visibility and session-adjacent hints, but not to E4 tool-invocation conformance or verified tool results.

Taken together, the preceding subsections establish four results: recurrent raw control-plane traffic (Cursor-specific), fragment-level support recovery in BidiAppend (Cursor-specific), stronger session-level support recovery in RunSSE (Cursor-specific), and MCP setup traces (observed in both Cursor and GPT-side runs). The next question is whether that combination remains distinctive when compared with the GPT-

TABLE XV
SUPPORT-DERIVED CROSS-LAYER ALIGNMENT CUES BETWEEN BIDIAPPEND AND RUNSSE IN THE TWO CURSOR(AUTO) RUNS.

Recorded run	BidiAppend raw count and grouping	Field-2/request-id agreement	Field-3 pattern	RunSSE request-set and session alignment	Interpretation
`Cursor(Auto) Test1`	2,155 raw BidiAppend files distributed across 11 request ids and one session; 138 decoded derivatives.	2,155/2,155 agreement between wrapper field 2 and the request identifier.	Top-level field 3 appears in 2,145/2,155 files, ranges from 1 to 729, resets at request boundaries, and is almost monotonic within each request.	11 raw and 11 decoded RunSSE files; 11/11 exact request-set match; one-session alignment across the run.	The reviewed support material is consistent with field 2 acting as the request identifier and field 3 acting as a request-scoped append sequence, while RunSSE offers a higher-level event-stream view over the same request set.
`Cursor(Auto) Test2`	1,866 raw BidiAppend files distributed across 2 request ids and one session; 2 decoded derivatives.	1,866/1,866 agreement between wrapper field 2 and the request identifier.	Top-level field 3 appears in 1,864/1,866 files, ranges from 1 to 2122, resets at request boundaries, and is almost monotonic within each request.	2 raw and 2 decoded RunSSE files; 2/2 exact request-set match; one-session alignment across the run.	The same support-guided pattern reappears with heavier fragmentation, favoring a request-scoped append-fragment reading for BidiAppend and a higher-level event-stream view for RunSSE.

TABLE XVI
WORKED RAW TRACE FOR NOTION MCP SESSION ESTABLISHMENT IN THE REVIEWED DIRECT-EVIDENCE CORPUS.

Trace phase	Visible field class	Observed value summary	Bound of what it shows
Request line	method and path	`POST /mcp` at `2026-03-05T02:36:56.4198217Z`.	Transport setup request exists.
Request body	JSON-RPC method	`initialize` with client capability advertisement.	Client announces setup capabilities only.
Response metadata	status and content type	`200 OK`, `Content-Type: text/event-stream`, and an `mcp-session-id` header redacted as `SESSION-01`.	Connection accepted and session header exposed.
Event-stream body	advertised server result	Event-stream payload advertises `protocolVersion`, `tools`, `resources`, `completions`, and `serverInfo.name="Notion MCP"`.	Capability advertisement is visible.
Limit row	absent downstream result	No later tool-result row is included in this worked trace.	Setup does not prove successful tool completion.

side runs. The next three subsections answer that question by moving from same-prompt comparison, to within-Cursor prompt variation as an internal control, and then to GPT-route differentiation. Prompt and plan alignment remains support-only throughout, and no comparison in this sequence is used to claim internal behavioral equivalence.

G. Same-Prompt Comparison Across Cursor and GPT IDE-Extension Runs

Prompt and plan artifacts (support-only) align `Cursor(Auto) Test1`, `GPT 5.4 IDE Ext. Test1`, and `GPT(Codex 5.3) IDE Ext. Test1` on the same mini Celery workload, so the raw corpus can be compared at the host-family and transport-surface level. Table XVII summarizes what stays shared and what diverges. The shared host families coexist with clear surface divergence: Cursor remains BidiAppend-centered, whereas the GPT-side runs expose additional `chatgpt.com` control-plane and `ab.chatgpt.com` experimentation or metrics surfaces. The same workload supports a bounded transport-surface comparison; it does not prove identical internal behavior across ecosystems.

TABLE XVII
SAME-PROMPT COMPARISON ACROSS CURSOR(AUTO) TEST1, GPT 5.4 IDE EXT. TEST1, AND GPT(CODEX 5.3) IDE EXT. TEST1.

Comparison axis	Observation
Workload alignment (support only)	The same mini Celery task appears in prompt and plan artifacts across Cursor(Auto) Test1, GPT 5.4 IDE Ext. Test1, and GPT(Codex 5.3) IDE Ext. Test1.
Shared raw hosts	All three runs expose `api2.cursor.sh`, `mcp.notion.com`, and `chat.openai.com`.
Cursor(Auto) Test1 surface	BidiAppend dominates the raw mix, and no comparable `chatgpt.com` or `ab.chatgpt.com` family appears at GPT-like scale. Cursor-side support material separates BidiAppend fragment recovery from stronger RunSSE session-level recovery.
GPT 5.4 IDE Ext. Test1 surface	Adds `chatgpt.com` usage polling and `ab.chatgpt.com` experimentation or metrics traffic; recovered support material separates polling, telemetry, and experimentation surfaces.
GPT(Codex 5.3) IDE Ext. Test1 surface	Adds `chatgpt.com` response-channel traffic and `ab.chatgpt.com` metrics or experimentation traffic; GPT-side support material, together with direct response-channel rows, supports a different route reading.
Interpretation	Shared workload context is support-only. The direct evidence supports a host-family comparison, not internal equivalence, identical action graphs, or equivalent semantic visibility.

Support-only Cursor derivatives recover richer semantic classes than the raw host-family counts reveal, but the recovery strength differs sharply across BidiAppend and RunSSE and across Test1 and Test2. BidiAppend derivatives recover recurring prompt, skill, plan, settings, and code-related fragments, whereas RunSSE derivatives preserve a stronger planning-to-implementation-to-reporting session path. Within the same mini-Celery workload, the Cursor side therefore combines a BidiAppend-centered raw surface with a split support-guided recovery profile rather than a single transcript-like stream.

The GPT routes are visibly different from that Cursor path. For `GPT 5.4 IDE Ext. Test1`, recovered `chatgpt.com` material follows a startup burst across `accounts/check`, `codex/models`, `tasks/list`, `usage`, and `environments`, including an account-versus-usage plan mismatch and model definitions with `base_instructions`. Recovered `chat.openai.com` and `ab.chatgpt.com` material from the same run preserve telemetry and experimentation or metrics surfaces.

For `GPT(Codex 5.3) IDE Ext. Test1`, recovered `chatgpt.com` material is more generation-heavy, with 134 successful `POST /backend-api/codex/responses` rows, one transient `503` followed by recovery, 72,424 `response.function_call_arguments.delta` events, and generated-file traces such as `docker-compose.yml`, `mini_celery/models.py`, and `architecture_report.md`.

H. Different-Prompt Comparison Within Cursor(Auto)

Prompt and plan files (support-only) show that `Cursor(Auto) Test1` uses the mini Celery workload, whereas `Cursor(Auto) Test2` uses the netprof workload. Across that prompt change, the raw control-plane family remains stable, making the shift in support-visible semantics and decode footprint easier to compare. Table XVIII summarizes this within-Cursor comparison.

TABLE XVIII
DIFFERENT-PROMPT COMPARISON WITHIN THE TWO RECORDED CURSOR(AUTO) RUNS.

Comparison axis	Observation
Prompt source (support only)	Prompt and plan artifacts indicate that `Cursor(Auto) Test1` uses the mini Celery task, while `Cursor(Auto) Test2` uses the netprof task.
Stable raw signals	Both runs retain `api2.cursor.sh` dominance, repeated BidiAppend traffic, and visible Notion MCP traces.
Support-visible shift	Test1 BidiAppend and RunSSE derivatives recover mini queue planning, implementation, and code/session flow, whereas Test2 derivatives recover netprof planning, architecture, traffic-profiler fragments, and a clearer two-stage execution/verification flow.
Decode footprint	Test1 yields 138 decoded BidiAppend files and 11 decoded RunSSE files, whereas Test2 yields 2 decoded BidiAppend files and 2 decoded RunSSE files.
Interpretation	Prompt change aligns with a difference in the task content recovered from the support-only derivatives, but it does not by itself explain every host-count or telemetry difference.

Across the two Cursor(Auto) runs, the BidiAppend-centered raw control-plane shape stays stable, but the support-visible semantic content changes at both the fragment and session layers. The prompt change supports a semantic-shift reading, though other factors (session length, environment) may also contribute.

Recovered derivative material makes that semantic shift more concrete. In Test1, BidiAppend fragments and RunSSE decoded derivatives recover material consistent with a mini queue, scheduler, worker, API, integration-testing, validation, and report-writing flow, whereas in Test2 the same two layers recover material consistent with an offline traffic-collector or netprof workload, plan frontmatter, architecture overview, Mermaid-form design material, and implementation-and-fix flow. The BidiAppend support-visible shape also changes: Test1 yields many overlapping support windows across recurring skill and settings families, whereas Test2 collapses from 1,866 raw shards into two large decoded aggregates. The RunSSE layer remains clearer than BidiAppend in both runs, but it still stops short of a direct transcript.

I. GPT IDE-Extension Route Differentiation

With the same-prompt comparison across Cursor and GPT and the within-Cursor prompt control in place, the remaining RQ3 question is how the GPT-side runs broaden the observable route surface beyond Cursor-style control-plane RPCs. At corpus scale, the direct raw census shows a broader GPT-side endpoint-family set without recovering a backend workflow: two runs contribute three `GET /backend-api/codex/responses` rows with `101 Switching Protocols`; two runs contribute twenty-five `GET /backend-api/wham/usage` rows with `200 OK`; three runs contribute 464 `POST /ces/v1/telemetry/intake` rows with `202 Accepted`; and two runs contribute 62 `POST /otlp/v1/metrics` rows with `202 Accepted`.

The route readings below are support-only interpretations and are not treated as backend reconstruction or direct evidence of internal execution.

For the `GPT 5.4 IDE Ext. Test1 chatgpt.com` route, reviewed support material shows a short startup burst that touches `accounts/check`, `codex/models`, `tasks/list`, `usage`, and `environments` before later response-channel setup and repeated polling. The reviewed support material also shows a large `codex/models` outlier with `base_instructions`, task-card metadata from `tasks/list`, and an account-versus-usage plan-view mismatch in which `accounts/check` and `usage` report different plan states. Taken together, the startup burst, the `codex/models` outlier, and the plan-state mismatch are consistent with a broader control-plane preparation pattern rather than a single transcript-like stream.

The reviewed `chat.openai.com` support material traces a distinct telemetry pattern. The captured payloads consistently frame themselves as `codex-extension` operational telemetry, and they are dominated by 1,689 logged messages with 1,620 warnings, repeated `IpcClient` notices tied to `thread-stream-state-changed`, pseudo-URL fetch failures, repeated `skills/list` lookups, local database lock or slow-insert messages, and PowerShell shell-snapshot limits. This pattern contributes runtime and persistence-state visibility rather than conversational content.

The reviewed `ab.chatgpt.com` support material maps a separate initialization, registration, and metrics pattern. The preserved sequence combines 3 `/v1/initialize`, 4 `/v1/rgstr`, and 32 `/otlp/v1/metrics` uploads; exposes approximately 35 feature gates together with 16 dynamic configs and 6 layer configs; and records gate-exposure or product-event registration alongside cadence-like metrics flushes. The same records also surface websocket event families, `tool = shell_command`, `sandbox_policy`

= danger-full-access, model values, and token-usage classes.

The Codex 5.3 chatgpt.com route differs again in reviewed support material. Its recovered support stream is dominated by 134 successful POST /backend-api/codex/responses rows, one transient 503 followed by later recovery, and 72,424 response.function_call_arguments.delta events. Generated-file traces such as docker-compose.yml, mini_celery/models.py, and architecture_report.md become more visible at the response-channel layer than in the polling-heavy GPT 5.4 route. These support-guided readings distinguish several GPT-side route patterns from the Cursor control-plane path, but they do not amount to a backend reconstruction or direct evidence of full internal behavior.

J. Ancillary Coexistence Surfaces (Context Only)

Ancillary marketplace, package, and IDE telemetry requests are retained as background capture-path context only and are excluded from the RQ3 direct-evidence positives; Appendix C Table C5 records representative families and counts.

K. RQ3 Synthesis: Directly Visible Signals, Support-Guided Comparisons, and Residual Gaps

Because the corpus includes both raw wrappers (direct evidence) and decoded derivatives (support-only material), the case study supports a two-layer answer relative to the Section IV baseline.

Direct-evidence answer: At the raw level, the proxy-path corpus reveals recurring setup, control-plane, response-channel, usage, and telemetry surfaces, but it does not provide the joined action records required for reconstruction-ready attribution. These signals illuminate parts of T2 and T4, but they do not show that the shared join keys defined in Table VIII (agent_id, session_id, trace_id/span_id) propagate consistently across the different data sources. T3, T5, and T6 therefore remain unevidenced in the direct corpus.

Support-only interpretation: Derived material recovers additional semantic structure, but those recoveries remain secondary to the raw corpus and are not treated as direct evidence. On the Cursor side, the BidiAppend derivatives recover recurring prompt, skill, plan, settings, and code-related fragments, while the RunSSE derivatives provide a stronger—though still incomplete—session-level view of planning, implementation, validation, reporting, and verification. Together they support a two-level interpretation in which BidiAppend preserves request-fragment views and RunSSE preserves a higher-level session view. On the GPT side, the support material distinguishes startup or polling, telemetry, initialization or metrics, and generation-heavy response patterns rather than a single transcript-like stream. These readings remain interpretive and do not supply direct evidence for T3, T5, or T6.

The current corpus still does not provide a complete semantic decode of every framed payload, a full prompt-to-request causal chain, direct policy-decision evidence, approval-workflow evidence, integrity-control evidence, or a backend reconstruction. Accordingly, Section V should be read as a record of what this proxy path can directly show, together with a list of claims the corpus cannot support, rather than as a complete execution ledger. Tables XIX and XX summarize the route patterns and the claims the present corpus cannot support.

Because the case study remains proxy-path dependent and leaves several evidence classes absent, the next section turns to the conditions that limit both the Section V findings and the framework's applicability.

VI. Limitations

Section V established what the current proxy path can and cannot show in the reviewed corpus. This section explains the conditions that limit that result: transport visibility, interception feasibility, identity integration, schema heterogeneity, and dataset scope.

First, the encrypted and multiplexed transport constraints documented in Sections I-A and III—TLS 1.3, HTTP/2, QUIC/HTTP/3, and ECH [1], [2], [3], [4], [5]—limit boundary-only inference to coarse metadata. Consistent with the visibility-baseline result in Section V, network-only observation fills only part of T4.

Second, MITM-style interception is treated here only as an optional upper-bound measurement posture, not as the framework default. In practice, it increases L7 observability only when a client trusts the interception CA [66] and no certificate pinning or equivalent control blocks decryption [66], [67]. Even under that posture, QUIC/HTTP/3 remains a separate transport path rather than a directly comparable proxy-visible HTTP/2 case.

Third, attribution quality depends on cross-system identity integration. Workload-identity primitives such as SPIFFE/SPIRE provide a technical basis for binding workload instances and credentials [9], [10], [11]. For the reconstruction objective used here, however, consistent propagation across runtimes, gateways, and delegated services remains a practical deployment condition rather than an assumption of this study.

Fourth, existing tracing and GenAI conventions provide useful standardization anchors [19], [39], [40], but they do not by themselves guarantee a common schema across heterogeneous products. An additional normalization layer is therefore required, and this requirement is a framework conclusion rather than an established practice.

Finally, the dataset scope is limited to one small IDE proxy corpus. Methodologically, Section V instantiates only the M1/M2 visibility-baseline evaluation posture: a clearly bounded direct-evidence corpus that depends on proxy-path interception and states explicit non-claims rather than producing direct empirical scores. The paired empirical validation dimensions M3/M4 remain out of scope because they require baseline-versus-instrumented runs with request-level transfer ground truth outside the current dataset.

TABLE XIX
GPT IDE-EXTENSION ROUTE TAXONOMY IN SECTION V, USING DERIVED MATERIAL ONLY AS SUPPORT.

Route	Dominant signals	Preserved support-visible facts	Analytical use
GPT 5.4 chatgpt.com	Startup burst, usage polling, model and task metadata.	accounts/check, usage, tasks/list, environments, codex/models, task-card metadata, and base_instructions, plus an account-versus-usage plan-view mismatch.	Indicates a broader control-plane preparation path before or alongside response-channel setup.
GPT 5.4 chat.openai.com	Telemetry framed as codex-extension.	1,689 logged messages with 1,620 warnings, repeated IpcClient and thread-stream-state-changed, pseudo-URL failures, DB lock or slow-insert messages, PowerShell shell-snapshot limits, and skills/list.	Indicates a runtime and persistence-state route rather than prompt or completion content.
GPT 5.4 ab.chatgpt.com	Initialization, registration, and metrics cadence.	3 /v1/initialize, 4 /v1/rgstr, 32 /otlp/v1/metrics, feature gates in the mid-30s, 16 dynamic configs, 6 layer configs, gate exposure, product-event registration, websocket event families, and tool or sandbox attributes.	Indicates experimentation and metrics surfaces that expose IDE-extension feature and runtime measurement structure.
GPT(Codex 5.3) IDE Ext. Test1 chatgpt.com	Generation-heavy response channel.	134 successful response rows, one 503 followed by recovery, 72,424 response.function_call_arguments.delta events, generated filenames, and a repeated plan-view mismatch.	Indicates a route that is more directly shaped by streamed generation and file-creation traces than the polling-heavy GPT 5.4 path.

TABLE XX
EXPLICIT SECTION V NON-CLAIMS REQUIRED BY THE CURRENT DIRECT-EVIDENCE BOUNDARY.

ID	Non-claim register
NC-01	**Avoid:** Successful end-to-end Notion tool use. **Why out:** Setup and OAuth-discovery traces stop short of a verified tool result. **Safe replacement:** State that the corpus shows MCP setup and discovery only.
NC-02	**Avoid:** Complete prompt-to-request execution graph. **Why out:** Join-like fields exist, but no normalized execution ledger closes the whole chain. **Safe replacement:** State that action-level causality remains incomplete.
NC-03	**Avoid:** Complete semantic decode of every BidiAppend payload. **Why out:** Many raw rows remain transport-framed, while support derivatives recover only part of the semantics. **Safe replacement:** State that transport recurrence is directly visible and that support derivatives recover only part of the semantic content.
NC-04	**Avoid:** Claims of policy-decision, approval-workflow, or evidence-integrity guarantees. **Why out:** No first-class policy-decision, approval-workflow, or integrity-control raw row is present. **Safe replacement:** State that those evidence classes are absent from the current corpus.
NC-05	**Avoid:** Silent proxy path proves behavior absence. **Why out:** The dataset is capture-path dependent. **Safe replacement:** State that uncaptured paths remain unverified.
NC-06	**Avoid:** A 101 handshake proves response content or success. **Why out:** The visible raw row ends at protocol upgrade. **Safe replacement:** State that the row proves channel establishment only.
NC-07	**Avoid:** The same prompt proves identical internal agent behavior across Cursor and GPT. **Why out:** Prompt identity is support-only context, and the raw surface mix differs across ecosystems. **Safe replacement:** State that the same workload permits a bounded host-family comparison only.
NC-08	**Avoid:** The different Cursor(Auto) prompt explains every run-level traffic difference. **Why out:** Prompt-linked differences in the task content recovered from support derivatives are visible, but host-count and telemetry differences may also reflect session length or environment. **Safe replacement:** State that prompt change aligns with a difference in the task content recovered from the support-only derivatives while full causality remains unresolved.

A. Discussion: tradeoffs and operational constraints

These limitations not only constrain the study's claims but also raise practical questions about framework adoption. Even if the minimum interoperable schema were adopted, several operational tradeoffs would remain.

Privacy minimization vs. auditability: Section IV treats raw content as optional and prioritizes REQUIRED join fields; this reduces log-exposure risk but can reduce forensic detail in some incident classes.

Coverage vs. operational cost: increasing T1–T6 coverage increases collection, retention, and normalization cost; deployment planning must therefore include pipeline and storage capacity.

Conditional interception and schema normalization: Tracing and GenAI conventions provide useful standardization anchors [19], [39], [40]. Schema normalization nevertheless remains an integration choice across heterogeneous products. Interception feasibility also depends on local certificate trust and protocol path [66], [67]. Under the minimum interoperable schema proposed here, reconstruction quality therefore depends on selective interception where feasible, schema normalization across products, and disciplined join-key propagation.

These are operational deployment consequences of the framework. The next subsection turns from those consequences to the methodological validity bounds of this study.

B. Threats to validity

Construct validity: artifact-based inference can misclassify capability boundaries when documentation is incomplete or when preserved artifacts underdescribe deployed behavior. Section III reduces this risk through S/D/X evidence grading. X-grade signals remain contextual cues only and do not anchor requirements.

Internal validity: coding and interpretation were performed without independent inter-rater validation, so coding reliability cannot be independently checked. Separately, the proxy-path evidence structure in Section V limits causal attribution of observed gaps, because absent rows may reflect capture dependence rather than behavioral absence.

External validity: The observed visibility pattern may vary across BP1–BP5 deployment structures and across heavily customized enterprise stacks. This study therefore supports a scoped MPR framework and a bounded visibility-baseline result, not general prevalence claims across deployments.

These bounds therefore exclude universal deployment claims and deterministic causal reconstruction beyond the recorded evidence.

VII. Future Work

Four directions follow directly from unresolved items in Sections III–V.

First, multi-deployment studies should evaluate the M1/M2 visibility-baseline dimensions across BP1–BP5 with the same REQUIRED field checklist and join criteria. The M3/M4 paired empirical validation dimensions should then be instantiated through baseline-versus-instrumented runs with request-level transfer ground truth.

Second, future work should test whether the graph-causality field set proposed here—motivated by adjacent Trace Context correlation [18], GenAI span conventions [59], and token-exchange materials [60]—remains stable across EP3–EP5 boundaries. Request_id-only fallback should likewise be tested under partial tracing loss to determine when linear joins remain acceptable and when graph-causal reconstruction degrades materially.

Third, future work should test which combinations of minimization, redaction, logging, and monitoring preserve REQUIRED joins while reducing log-to-leak exposure. Existing LLM and agent security guidance motivates that problem setting [6], [7]. Broader AI governance guidance also reinforces the need for monitoring and governance [8]. Neither source class, however, specifies which join-preserving logging combination to adopt.

Fourth, the framework should be evaluated on non-adversarial failure incidents (e.g., hallucination-driven tool misuse) to confirm that the REQUIRED field set remains sufficient for evidence-based root-cause triage across BP1–BP5.

VIII. Conclusion

Under the encrypted and multiplexed transport conditions examined here, the gap between boundary-visible metadata and execution-level semantics is structural. For agent incidents under those conditions, audit-ready reconstruction therefore cannot be obtained from network-only observation alone: the signals needed for accountable attribution (T1), tool-path causality (T2), context lineage (T3), exposure accounting (T4), policy/approval evidence (T5), and evidentiary trust (T6) must remain joinable across layers.

This study addresses that reconstruction limitation by deriving a technology-agnostic minimum practical requirements framework for session-scoped post-incident reconstruction and audit-ready attribution. The framework organizes the problem across five observation and enforcement points, six telemetry axes, and a minimum interoperable schema of normalized event types with shared correlation keys. It is intended as a scoped interoperability contract under the stated deployment and evidence assumptions, not as a universal proof of minimality. Within this framework, deterministic cross-source reconstruction requires that participating systems propagate the required join keys, preserve comparable event timestamps, record identifier translations where boundaries rewrite them, and make evidence gaps explicitly detectable.

The bounded IDE-based proxy corpus reinforces both the need for and the limits of this approach. Even across four recorded runs and over 8,000 raw wrappers, proxy-path interception reveals recurring setup, control-plane, and partial response/telemetry surfaces, but context-lineage records (T3), policy-decision or approval-path records (T5), and integrity-control records (T6) remain entirely unevidenced in the direct corpus, and stable end-to-end join propagation is not demonstrated. The empirical result is limited to what this proxy path directly shows, together with explicit non-claims; it is not a complete execution ledger or a full validation of all reconstruction conditions.

The main implication is practical. Enterprise agent-security observability should be evaluated less by product feature lists than by whether a deployment emits the minimum joinable evidence needed to answer the scoped forensic questions with defensible confidence. In that sense, the contribution of this study is not a new detector or benchmark, but a schema-level framework for designing, auditing, and comparing telemetry pipelines for agent incident reconstruction.

Appendix orientation. The appendices provide audit and support material for claims already stated in the main body. Appendix A records the verified source harvest list and the field-level evidence matrix (Table A2) that makes RQ2 auditable. Appendix B documents supplementary source verification notes. Appendix C provides sanitized artifact excerpts and ancillary surface tables from the case study. Appendix D decomposes BP1–BP5 into scenario-level detail (S1–S13). Appendix E formalizes the axis-removal and substitute-signal analysis that justifies REQUIRED-field promotion. No appendix introduces new claims or argumentative burden beyond what the main body establishes.

APPENDIX A
PRIMARY SOURCE HARVEST LIST

Table A1 below forms one multi-page harvest list of the verified sources cited in this study and the types of claims each source supports. List order follows the appendix audit sequence, while bracketed bibliography numbers preserve the manuscript-wide citation numbering.

TABLE A1
VERIFIED SOURCES USED IN THE MANUSCRIPT.

List No.	Source type [Bib. No.]	What it supports in this study (example claims)
1	Standard (IETF/W3C) [1]	Defines TLS 1.3 encryption properties and reduced in-path visibility assumptions.
2	Standard (IETF/W3C) [3]	Defines QUIC transport properties relevant to multiplexing and metadata visibility limits.
3	Standard (IETF/W3C) [2]	Defines HTTP/2 multiplexing and stream semantics that complicate flow-level attribution.
4	Standard (IETF/W3C) [4]	Defines HTTP/3 mapping over QUIC and implications for encrypted semantics and multiplexing.
5	Standard (IETF/W3C) [5]	Specifies ECH behavior and handshake-privacy design relevant to destination-visibility limits.
6	Industry security guidance [6]	Threat taxonomy for LLM applications; supports risk framing (OWASP) and control categories.
7	Industry security guidance [7]	Threat taxonomy for agentic applications; supports agent-specific (OWASP) risk framing.
8	Government framework (NIST) [8]	Governance and risk-management framework for AI; supports monitoring and measurement motivation.
9	Vendor/organization [11]	Introduces SPIFFE concept of workload identity and trust domain boundaries.
10	Vendor/organization [9]	Defines SPIFFE ID format; supports identity naming and binding discussions.
11	Vendor/organization [10]	Describes SPIRE as an implementation for issuing/verifying SPIFFE identities.
12	Academic publication [12]	Introduces the 'confused deputy' concept used in tool-delegation threat modeling.
13	Vendor/organization [23]	Defines MCP as a tool connectivity protocol and supports tool-call semantics.
14	Vendor/organization [24]	Explains TLS interception mechanics for controlled measurement and its limitations.
15	Preprint (arXiv) [13]	Prompt injection attacks study; supports general prompt-injection risk and mitigation discussion.
16	Preprint (arXiv) [15]	End-to-end evaluation of indirect prompt manipulations in RAG; supports RAG-focused threat claims.
17	Preprint (arXiv) [16]	AgentPoison; supports risks from poisoning memory/knowledge bases in agent systems.
18	Vendor/organization [17]	NCSC analysis; supports residual-risk framing for prompt injection.
19	Vendor/organization [14]	CETaS expert analysis; supports characterization of indirect prompt-injection risks.
20	Vendor/organization [25]	Model Armor overview; supports managed model firewall/guardrail discussion.
21	Vendor/organization [26]	Prompt Shields/jailbreak detection; supports provider-side mitigation and detection examples.
22	Vendor/organization [20]	MCP security best practices; supports gateway/tool-boundary recommendations.
23	Vendor/organization [27]	Example MCP gateway source used to ground EP4 (tool-path) control mechanisms.
24	Vendor/organization [28]	Vendor source on MCP threat detection; supports feasibility and types of MCP-focused protections.
25	Vendor/organization [29]	Gateway DLP feature source; supports boundary-scanning capability and documented DLP-action surfaces.
26	Vendor/organization [30]	Purview AI source; supports compliance/oversight framing for AI activity monitoring.
27	Vendor/organization [19]	OpenTelemetry logs spec; supports log-based event representation and interoperability claims.
28	Vendor/organization [31]	OpenAI Agents SDK tracing; supports feasibility of SDK-level tracing for agent workflows.
29	Vendor/organization [32]	Primary announcement framing MCP goals and ecosystem context.
30	Vendor/organization [21]	MCP authorization specification; supports OAuth-based authorization-flow claims.
31	Standard (IETF/W3C) [33]	Best current practice for OAuth 2.0 security; supports delegation-related security guidance and mitigation framing.
32	Vendor/organization [34]	Example of commercial MCP gateway authorization integration (OAuth2 mediation).
33	Vendor/organization [35]	LiteLLM proxy configuration; supports proxy/gateway observability and OTel export patterns.

Continued on next page

TABLE A1 (continued)
VERIFIED SOURCES USED IN THE MANUSCRIPT.

List No.	Source type [Bib. No.]	What it supports in this study (example claims)
34	Open-source [36]	OpenLLMetry instrumentation; supports open-source OTel-compatible tracing for LLM apps.
35	Standard (W3C) [18]	Defines Trace Context (traceparent/tracestate) used for cross-system correlation identifiers.
36	Vendor/organization [22]	Gateway observability source used to ground logging/tracing claims for MCP mediation.
37	Vendor/organization [37]	AWS Bedrock Guardrails; supports provider guardrails and content policy-enforcement discussion.
38	Vendor/organization [38]	Audit log retention policies; supports retention controls as part of evidence integrity requirements.
39	Vendor/organization [39]	OpenTelemetry GenAI attribute registry; supports standardized attributes relevant to model/tool telemetry used in Table A2.
40	Vendor/organization [40]	OpenTelemetry GenAI metrics; supports quantitative telemetry for evaluation and monitoring.
41	Vendor/organization [41]	CloudTrail log file validation; supports log integrity validation as evidence control.
42	Government framework (NIST) [42]	NIST SP 800-53 Rev. 5; supports audit event selection and audit record content framing (AU-2/AU-3).
43	Standard (W3C) [43]	PROV-DM; supports provenance model alignment for lineage/context semantics.
44	Industry technical report [44]	Dapper tracing design; supports low-overhead instrumentation framing for distributed correlation.
45	Preprint (arXiv) [45]	CamFlow provenance capture; supports practical provenance capture with explicit overhead/feasibility considerations.
46	Preprint (arXiv) [46]	CamQuery runtime provenance analysis; supports selective/tailored analysis and overhead-aware provenance processing.
47	Standard (CloudEvents) [47]	Defines required event attribute `type`; supports canonical event-type mapping in Table A2.
48	Vendor/organization [48]	Defines session semantic-convention attributes (e.g., `session.id`, `session.previous_id`) used for session_id mapping in Table A2.
49	Standard specification [49]	Defines JSON-RPC 2.0 request `method`; supports canonical tool-action mapping in Table A2.
50	Standard (OpenID Foundation) [50]	Defines `sub`/`iss`/`aud` claims; supports identity and token fields in Table A2.
51	Standard (OASIS) [51]	Defines policy and decision semantics; supports `policy_id`/`decision` mappings in Table A2.
52	Open-source documentation [52]	Documents decision-log schema including the decision identifier and query-input context. The later actor-context interpretation used in Table A2 is a study-owned normalization note rather than a source-backed extension of this row.
53	Vendor/organization [53]	Defines `network.peer.address`/`network.peer.port`; supports destination-field canonicalization in Table A2.
54	Standard (IETF) [54]	Defines DNS terminology relevant to FQDN semantics; supports optional `dst.fqdn` interpretation in Table VIII and Appendix A discussion.
55	Online registry standard [55]	Defines IPFIX information elements (e.g., `octetDeltaCount`); supports `bytes_out` mapping in Table A2.
56	Metadata standard [56]	Defines core metadata elements (e.g., identifier/type); supports context-field canonicalization anchors in Table A2.
57	Standard (IETF) [57]	Defines URI generic syntax; supports `context.ref` form and normalization in Table A2.
58	Standard (IETF) [58]	Defines digest field syntax/semantics; supports `context.digest` as structured integrity metadata in Table A2.
59	Vendor/organization [59]	Defines GenAI span conventions and tool-call attributes. The broader parent/child interpretation used later is a study-owned synthesis over the wider tracing model rather than a claim sourced from this row alone.
60	Standard (IETF) [60]	Defines OAuth 2.0 token exchange semantics; supports delegated-path lineage and token-exchange evidence for `delegation.chain_id` in Table A2 and Table VIII.
61	Vendor/organization [61]	Model Armor logging configuration; supports documented evaluation-result and metadata fields for sanitize operations.
62	Vendor/organization [62]	Azure AI Content Safety Shield Prompt API; supports prompt-attack analysis result fields for request-level policy output claims.
63	Vendor/organization [63]	Bedrock ApplyGuardrail API reference; supports `action`/`actionReason`/`assessments`/`usage` response fields.
64	Vendor/organization [64]	Bedrock token-count documentation; supports provider-exposed token-usage metrics.
65	Vendor/organization [65]	Cloudflare AI Gateway logging; supports request-level token-usage and DLP-action log claims.
66	Open-source documentation [66]	mitmproxy certificate documentation; supports interception-CA trust and certificate-pinning feasibility limits.
67	Open-source documentation [67]	mitmproxy domain-bypass documentation; supports selective interception limits under pinning or incompatible paths.

a) Primary-source method note: This appendix records the detailed validation and promotion controls summarized in the survey's evidence-strength discussion in Section III. Coding was performed conservatively by a single reviewer, ambiguous mappings were downgraded rather than resolved by inference, and REQUIRED-field promotion follows a fixed rule over alias canonicalization, recurrence support, FQ/axis/event mapping, and the scoped substitute test in Appendix E.

b) Canonicalization rules for Table A2:

- `traceparent` is canonicalized to `trace_id`/`span_id` (W3C Trace Context form), while `X-Request-Id` is canonicalized to `request_id`.
- `parentSpanId`, span parent linkage, and gateway-carried parent request identifiers are canonicalized to `parent_span_id` or `parent_request_id`, depending on whether W3C trace lineage is preserved.
- `X-Session-Id` is canonicalized to `session_id`.
- Provider- or gateway-specific tool-invocation handles are canonicalized to `tool.call_id`; OAuth token-exchange or delegated-execution lineage handles are canonicalized to `delegation.chain_id`.
- For recurrence comparison in Table A2, destination-related aliases may be grouped under a destination-oriented canonical mapping to `network.peer.*`, deprecated `net.sock.peer.*`, or gateway/proxy `egress_dest` forms. However, event-level conformance and M1 counting in Table VIII treat `dst.addr`, `dst.port`, and optional `dst.fqdn` separately wherever the normative checklist defines them separately.
- For Table A2 recurrence reporting, Appendix A source types are first mapped into four counting classes: standards (including RFC/W3C/OASIS/OpenID/CloudEvents specifications, metadata standards, and registry standards), government frameworks, documentary sources (vendor or organization documentation, industry security guidance, industry technical reports, academic publications, and preprints), and open-source specifications/documentation. Each row's "N domains" is the number of distinct counting classes represented by citations in that row's Evidence cell; Sx/Dy is the per-row count of (S)/(D) tags in that same cell.

c) Semantic-equivalence constraints for canonicalization: Canonical field-family membership in Table A2 does not imply value-level interchangeability. If issuer, scope, lifecycle, or observation-source semantics differ, the original qualifier MUST be preserved in the normalized record rather than collapsed away. Join anchors and lineage/reference pointers are not treated as interchangeable members of the same field family solely because they are often logged together. Accordingly, alias harmonization in this appendix is name normalization for comparison and recurrence accounting, not blanket semantic collapse across heterogeneous telemetry sources.

d) Legend for REQUIRED basis in Table A2: S/D/X denote evidence grade. *R* denotes recurrence-supported fields, assigned either by cross-domain repetition or by a standards-backed core primitive that is re-exposed by an independent operational artifact family. *N* denotes fields promoted by the scoped substitute test: removing the field makes at least one mapped forensic question unanswerable or materially weaker at the same evidentiary confidence, and no weaker substitute preserves that question. *R+N* denotes fields that satisfy both tests. *C* denotes fields that are conditionally REQUIRED only when the execution shape includes fan-out, repeated child actions, or delegated execution. "N domains" reports the number of supporting source-type classes represented in the row.

Table A2 at a glance: Table A2 reports field-family recurrence support, evidence grade, and promotion basis for the event-level REQUIRED fields in Table VIII.

TABLE A2

STUDY-OWNED FIELD/KEY EVIDENCE AND PROMOTION MATRIX FOR RQ2 (EVENT-LEVEL REQUIRED FIELDS IN TABLE VIII, E1–E7 ROWS).

Legend: Evidence grades use (S) Schema/Spec, (D) Documented, and (X) Not verifiable. In *Recurrence support*, N domains counts source-type classes and Sx/Dy counts the (S) and (D) tags in that row's Evidence cell. In *Basis*, R = recurrence-supported, N = substitute-test promotion, R+N = both, and C = conditional REQUIRED. Each row is a study-owned normalized field-family record. The Evidence column names only the source-backed primitive or adjacent artifact and carries all citations for that row. The Basis and Normalization / scope note columns are uncited study-owned framework commentary: they record recurrence counting, promotion decisions, canonical labels, and scope qualifications rather than source-native schema claims. When a row cites an adjacent artifact, that artifact is contextual only and does not define the retained canonical field-family label or promotion rule.

Field family (canonical)	Equivalent keys / aliases	Event types + EP(s)	Primitive / artifact evidence (Ref, grade)	Recurrence support	Basis (study-owned)	Normalization / scope note (uncited)
timestamp	time_unix_nano, ts, @timestamp	E1–E7; EP1–EP5	Timestamp or event-time fields [19] (S); audit-record content guidance [42] (D, adjacent context)	2 domains; S1/D1	R	Cross-system ordering-field family; emitted key names remain deployment-specific. Audit-record guidance is contextual only and does not define timestamp semantics or the canonical label.
event_type	msgType, category, kind, type (CloudEvents)	E1–E7; EP1–EP5	CloudEvents type attribute [47] (S); generic event-carrier example [19] (D, adjacent carrier)	2 domains; S1/D1	R	Normalized taxonomy slot across heterogeneous event families. Source-native event names remain deployment-specific.
agent_id	spiffe_id, workload_id, service identity, gen_ai.agent.id	E1–E6 + E7 (join-capable); EP1/EP2/EP3 (+EP5 when exported)	SPIFFE/SPIRE workload identifiers [9], [10], [11] (D); GenAI agent attribute [39] (S)	1 domain; S1/D3	N	Normalized identity family over workload and application-level agent surfaces. If only application-level IDs are emitted, the preserved value remains weaker than cryptographic workload proof.
owner	principal, subject	E1–E6 + E7 (join-capable); EP1/EP2/EP3 (+EP5 when exported)	OIDC iss+sub claims for accountable-principal attribution [50] (S)	1 domain; S1/D0	N	Framework-qualified owner family rather than a universal source-native schema. Account-level or tenant-level ownership semantics remain deployment-specific.
session_id	X-Session-Id, gen_ai.conversation.id, session.id	E1–E6 + E7 (join-capable); EP1–EP5	Session anchors such as gen_ai.conversation.id and session.id [39], [48] (S)	1 domain; S2/D0	R+N	Qualified session-anchor family; emitted values may be conversation-, thread-, or run-scoped. session.previous_id remains predecessor lineage rather than the current anchor.
trace_id/span_id OR request_id	traceparent, tracestate, X-Request-Id	E2/E4/E6 + E7 (join-capable); EP3/EP4/EP5	Standardized trace keys and log-correlation carriers [18], [19] (S)	1 domain; S2/D0	R+N	Request identifiers are fallback interoperability aids when tracing is unavailable. Deterministic policy-event joins remain a framework rule over those primitives.
parent_span_id OR parent_request_id	span parent linkage, parent request handle	E2/E4/E6 + E7 (graph-structured join-capable); EP3/EP4/EP5	Trace parent linkage [18] (direct). OpenTelemetry Logs may carry emitted linkage fields as generic log records [19] (adjacent carrier context). GenAI span conventions provide adjacent span-hierarchy context within instrumented GenAI traces [59] (adjacent context).	2 domains; S3/D0	C	Study-owned conditional graph-causality family. The parent_request_id fallback and the cross-event canonical family are framework-defined; linear request_id-only deployments retain weaker causality than explicit parent-child linkage.
gen_ai.operation.name	operation, action name	E2; EP3	[39] (S); [40] (S)	1 domain; S2/D0	R	SemConv-aligned model operation key.
model	model_id, provider model name	E2; EP3	Model-identity attributes and metrics guidance [39], [40] (S)	1 domain; S2/D0	R	Normalized model-identity family; concrete emitted keys remain provider- and runtime-specific.

Continued on next page

TABLE A2 (continued)

STUDY-OWNED FIELD/KEY EVIDENCE AND PROMOTION MATRIX FOR RQ2 (EVENT-LEVEL REQUIRED FIELDS IN TABLE VIII, E1–E7 ROWS), CONTINUED.

Legend: Evidence grades use `(S)` Schema/Spec, `(D)` Documented, and `(X)` Not verifiable. In *Recurrence support*, `N domains` counts source-type classes and `Sx/Dy` counts the `(S)` and `(D)` tags in that row's Evidence cell. In *Basis*, `R` = recurrence-supported, `N` = substitute-test promotion, `R+N` = both, and `C` = conditional REQUIRED. Each row is a study-owned normalized field-family record. The Evidence column names only the source-backed primitive or adjacent artifact and carries all citations for that row. The Basis and Normalization / scope note columns are uncited study-owned framework commentary: they record recurrence counting, promotion decisions, canonical labels, and scope qualifications rather than source-native schema claims. When a row cites an adjacent artifact, that artifact is contextual only and does not define the retained canonical field-family label or promotion rule.

Field family (canonical)	Equivalent keys / aliases	Event types + EP(s)	Primitive / artifact evidence (Ref, grade)	Recurrence support	Basis (study-owned)	Normalization / scope note (uncited)
direction	in/out, request/response role	E2; EP3	Adjacent request/response-role context in GenAI conventions and OTel logs [19], [39] (X, carrier/context only)	1 domain; S0/D0	N	Study-owned direction normalization used to separate request and response records; the cited sources provide adjacent carrier/context only and do not define a universal emitted `direction` key.
context.source_id	source_id, document_id	E3; EP4 (+EP3)	Metadata element identifiers [56] (S); provenance source identifiers [43], [45] (D)	2 domains; S1/D2	R+N	Normalized source-identifier family; literal field names and promotion remain deployment-specific.
context.type	source type, context class	E3; EP4 (+EP3)	Metadata and provenance type/class primitives [56] (S); [43], [45] (D)	2 domains; S1/D2	R	Normalized context-class family; deployment-specific naming remains expected.
context.ref	URI/path/reference pointer	E3; EP4 (+EP3)	URI syntax and provenance reference traversal [57] (S); [43], [45] (D)	2 domains; S1/D2	R	Reference-family projection over deployment-specific keys and paths.
context.digest	hash/checksum/ digest	E3; EP4 (+EP3)	Digest or checksum semantics [58] (S); provenance integrity context [43], [45] (D)	2 domains; S1/D2	R+N	Normalized digest family; materialization still varies by logging pipeline.
tool.name	function name, tool identifier	E4; EP4	MCP tool-boundary concepts and examples [20], [23] (D)	1 domain; S0/D2	N	Normalized capability identifier used to attribute action and outcome records to one tool surface.
tool.server_id	MCP server identifier, connector id	E4; EP4	Adjacent server-boundary context in MCP/runtime materials [20], [23] (D)	1 domain; S0/D2	N	Server-boundary normalization used when same-named tools are exposed behind distinct MCP servers or connectors.
tool.action	call/list/read/write action	E4; EP4	JSON-RPC `method` action key [49] (S); MCP/runtime action surfaces [23], [28] (D, adjacent context)	2 domains; S1/D2	R+N	Broader action vocabulary remains implementation-specific.
tool.call_id	tool invocation id, call handle, span-scoped tool call id	E4; EP4	JSON-RPC request IDs and GenAI span-scoped tool handles [49], [59] (S); MCP invocation context [23] (D)	2 domains; S2/D1	C	Conditional handle family for sibling tool actions or child outputs.
outcome	execution result/status (excluding E6 policy decisions)	E4; EP4/EP3	Adjacent execution-status or action outputs [22], [63] (D)	1 domain; S0/D2	N	Normalized execution-result family only; E6 policy decisions stay in the separate `decision` family.
identity.subject	`sub`, subject principal	E5; EP2	OIDC subject-claim semantics [50] (S)	1 domain; S1/D0	R	Subject projection into emitted logs remains deployment-specific.
issuer	`iss`, token/workload issuer	E5; EP2	OIDC issuer-claim semantics [50] (S)	1 domain; S1/D0	R	Issuer projection into workload or token logs remains deployment-specific.
token.event	issue/refresh/revoke	E5; EP2	Issuance and delegation security guidance [33] (D, adjacent context)	1 domain; S0/D1	N	Lifecycle normalization over issue, refresh, and revoke states; no cited source defines a literal emitted key.

Continued on next page

TABLE A2 (continued)

STUDY-OWNED FIELD/KEY EVIDENCE AND PROMOTION MATRIX FOR RQ2 (EVENT-LEVEL REQUIRED FIELDS IN TABLE VIII, E1–E7 ROWS), CONTINUED.

Field family (canonical)	Equivalent keys / aliases	Event types + EP(s)	Primitive / artifact evidence (Ref, grade)	Recurrence support	Basis (study-owned)	Normalization / scope note (uncited)
Legend: Evidence grades use `(S)` Schema/Spec, `(D)` Documented, and `(X)` Not verifiable. In *Recurrence support*, `N domains` counts source-type classes and `Sx/Dy` counts the `(S)` and `(D)` tags in that row's Evidence cell. In *Basis*, `R` = recurrence-supported, `N` = substitute-test promotion, `R+N` = both, and `C` = conditional REQUIRED. Each row is a study-owned normalized field-family record. The Evidence column names only the source-backed primitive or adjacent artifact and carries all citations for that row. The Basis and Normalization / scope note columns are uncited study-owned framework commentary: they record recurrence counting, promotion decisions, canonical labels, and scope qualifications rather than source-native schema claims. When a row cites an adjacent artifact, that artifact is contextual only and does not define the retained canonical field-family label or promotion rule.						
token.audience	`aud`, target service	E5; EP2	OIDC audience-claim semantics [50] (S)	1 domain; S1/D0	R	Audience projection into emitted logs remains deployment-specific.
delegation. chain_id	token-exchange lineage, delegated runtimechain, subagent chainid	E5 (+E4/E6/E7 when propagated); EP2/EP3/EP4/ EP5	Token-exchange lineage and delegated-path primitives [60] (S); issuance and authorization context [21], [33] (D)	2 domains; S1/D2	C	Cross-hop lineage family and conditional promotion remain framework-defined.
policy_id	rule_id, control_id	E6; EP3/EP4/EP5	XACML policy identifiers [51] (S); audit/vendor policy context [30], [42] (D, adjacent context)	3 domains; S1/D2	R+N	Cross-system policy-context normalization over deployment-specific fields.
decision	allow/block/ transform	E6; EP3/EP4/EP5	XACML decision semantics [51] (S); product-specific action outputs [63] (D, adjacent output)	2 domains; S1/D1	R+N	Cross-system decision mapping; approval-workflow state remains a deployment-specific extension.
decision_id	evaluation_id, decision handle	E6; EP3/EP4/EP5	[52] (S)	1 domain; S1/D0	N	Join handle for policy path reconstruction.
dst.addr	`network.peer. address` (preferred), `net.sock.peer. addr` (deprecated), egress destination IP	E7; EP5	[53] (S)	1 domain; S1/D0	R	Standardized network attribute family. Gateway destination fields, when present, are deployment-specific complements rather than primary semantics sources.
dst.port	`network.peer. port` (preferred), `net.sock.peer. port` (deprecated), egress destination port	E7; EP5	[53] (S)	1 domain; S1/D0	R	Standardized network attribute family. Gateway destination fields, when present, are deployment-specific complements rather than primary semantics sources.
bytes_out	bytes sent, transfer volume, egress bytes, `octet DeltaCount`	E7; EP5	[55] (S)	1 domain; S1/D0	R	Standardized byte-volume metric family; collection is path-dependent, and gateway or usage logs are illustrative complements rather than primary metric-semantics sources.

Note on approval-workflow fields: Approval-workflow fields are not included as baseline Table A2 field families because the surveyed corpus does not establish a universal recurrent approval-actor/reference schema. Where a deployment inserts a human approval gate, approval fields are treated as deployment-specific extension fields that must remain source-qualified.

APPENDIX B
SOURCE VERIFICATION NOTES

This appendix records a small set of source-stability and identifier clarifications performed to keep citations stable and mechanically verifiable (e.g., canonical URLs and corrected identifiers), without changing the technical claims.

The canonicalization choices in Appendix A are intended for field-family comparison and recurrence accounting. They do not imply that semantically distinct source values are collapsed into one interchangeable identifier.

For living documentation without a fixed release artifact, this study records both the access date and the exact snapshot or revision used. Where a page exposes a "Last updated" or comparable revision marker, that marker is carried into the bibliography.

Table B1 then lists the small set of sources for which canonical URLs or identifier clarifications materially affected citation stability. Each row in Table B1 is intentionally split into an artifact-identification surface and an uncited study-owned note surface. Citations appear only in the artifact column to identify retained artifacts for manuscript stability; they are not attached to the note text or used as evidentiary support for the note.

TABLE B1
SELECTED STUDY-OWNED SOURCE VERIFICATION NOTES (CANONICAL URLS AND IDENTIFIER CLARIFICATIONS).

Claim location	Retained artifact identifier	Retained canonical artifact / source	Uncited study-owned source-management note
ECH discussion in Section I-A	[5]	RFC 9849 / RFC Editor entry	Retained RFC Editor artifact for citation stability.
RAG indirect prompt manipulation in Section II-F1	[15]	arXiv:2408.05025	Identifier is aligned to the titled paper used in the discussion of indirect prompt manipulations.
Memory poisoning discussion in Section II-F1	[16]	arXiv:2407.12784	Identifier is aligned to the titled paper used in the discussion of agent memory poisoning.
Prompt-injection guidance in Section II-F1	[17]	NCSC-hosted PDF version	Managed archival artifact retained for stable access.
Model-call boundary controls in Section III-B2	[25]	Vendor documentation landing page	Canonical vendor landing page retained as the stable documentation anchor used in the manuscript.
Request-level policy/guardrail outputs and token/request telemetry in Section III-B2 and Section IV	[61], [62], [63], [64], [65]	Canonical API and logging documentation pages	Canonical API and logging artifacts retained as the stable anchors for product-specific output and telemetry references.
TLS interception limits in Section II-E and Section VI	[66], [67]	mitmproxy certificate/how-to pages	Certificate and how-to artifacts retained as stable references across living documentation changes.
MCP threat detection in Section III-B4	[28]	Vendor technical documentation page	Vendor technical documentation artifact retained as the stable anchor for this runtime-control surface.
Living specifications and documentation pages used across the manuscript	[9], [10], [11], [19], [20], [23], [31], [39], [40]	Canonical specification or documentation page	Snapshot-date and revision-marker tracking is used to keep these living artifacts mechanically stable in the manuscript.

APPENDIX C
CASE STUDY ARTIFACT EXCERPTS AND SCHEMA EXAMPLES

This appendix provides sanitized artifact excerpts, support-only derivative summaries, ancillary surface tables, and minimal schema examples that document Section V without adding new claims. The JSON snippets are illustrative schema examples rather than captured runtime logs.

A. Sanitized representative direct-evidence raw-anchor excerpts

This subsection reproduces a small set of sanitized direct-evidence anchors so that the Section V claims remain auditable without reproducing the full corpus.

Excerpt A1: BidiAppend raw wrapper excerpt.

```
POST https://api2.cursor.sh:443/aiserver.v1.BidiService/BidiAppend HTTP/1.1
Content-Type: application/proto
Connect-Protocol-Version: 1
X-Request-Id: REQUEST-01
X-Session-Id: SESSION-01
HTTP/1.1 200 OK
[payload omitted; raw wrapper stores protobuf/hex-framed content]
```

This excerpt is a transport-framed raw anchor only. Any prompt, plan, code, or session fragments discussed in Section V come from support-only derivative material rather than from direct semantic transparency in the raw wrapper itself. Appendix C-B inventories those semantic classes separately in support-only form.

Excerpt A2: MCP initialize raw wrapper excerpt.

```
POST https://mcp.notion.com:443/mcp HTTP/1.1
Content-Type: application/json
{"method":"initialize","params":{"protocolVersion":"2025-11-25",...}}
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

TABLE C1
DERIVED SEMANTIC CLASSES RECOVERED ACROSS THE FOUR CURSOR(AUTO) DERIVATIVE SETS, USED ONLY AS SUPPORT MATERIAL.

Derivative source	Recovered semantic classes	Bounded use in the case-study section
`Cursor(Auto) Test1` BidiAppend	Workload prompt, recurring skill-document families, description-like instruction blocks, settings-guide text, mini-queue plan paths, Python code fragments, and local path classes.	Indicates that framed BidiAppend payloads can still preserve stable semantic families across overlapping derivative windows.
`Cursor(Auto) Test1` RunSSE	System prompt, user workload, backend file-URL anchors, plan references, implementation updates, validation steps, tool-turn concentration, and report-writing requests.	Indicates a planning-to-implementation-to-reporting session path that is richer than the BidiAppend layer alone.
`Cursor(Auto) Test2` BidiAppend	Offline netprof workload prompt, skill-creation metadata, repeated settings-guide tails, and mixed path traces.	Indicates that netprof-related semantic classes survive even when the append layer collapses from many small shards to two large aggregates.
`Cursor(Auto) Test2` RunSSE	System prompt, plan frontmatter, architecture overview, Mermaid-form design material, implementation or fix traces, and verification flow.	Indicates a clearer planning-versus-execution split than in the mini-Celery run.

TABLE C2
DERIVED CROSS-LAYER SIGNALS LINKING BIDIAPPEND AND RUNSSE IN THE TWO CURSOR(AUTO) RUNS, USED ONLY AS SUPPORT MATERIAL.

Cursor run	BidiAppend signal set	RunSSE alignment	Bounded analytical use
`Cursor(Auto) Test1`	2,155 raw files across 11 request ids and one session; top-level field 3 in 2,145/2,155 files; field 2 agrees with the request identifier in 2,155/2,155 files; field 3 ranges from 1 to 729.	11 raw and 11 decoded RunSSE files; 11/11 exact request-set match; one shared session; each RunSSE request fans out into many append chunks.	Indicates that field 2 is best read as the request identifier, field 3 as a request-scoped append sequence or index, and BidiAppend as a lower-level append-fragment layer.
`Cursor(Auto) Test2`	1,866 raw files across 2 request ids and one session; top-level field 3 in 1,864/1,866 files; field 2 agrees with the request identifier in 1,866/1,866 files; field 3 ranges from 1 to 2122.	2 raw and 2 decoded RunSSE files; 2/2 exact request-set match; one shared session; the same fan-out pattern reappears under heavier fragmentation.	Indicates that the same layered relationship persists in the netprof run, with RunSSE preserving a higher event-stream session layer above many append fragments.

```
Mcp-Session-Id: SESSION-02
event: message
data: {"result":{"serverInfo":{"name":"Notion MCP"},...}}
```

Excerpt A3: Usage/account surface excerpt.

```
GET https://chatgpt.com:443/backend-api/wham/usage HTTP/1.1
Chatgpt-Account-Id: ACCOUNT-01
HTTP/1.1 200 OK
Content-Type: application/json
{"user_id":"ACCOUNT-01","plan_type":"pro",
"rate_limit":{"allowed":true,...}}
```

Excerpt A4: Telemetry intake excerpt.

```
POST https://chat.openai.com:443/ces/v1/telemetry/intake?... HTTP/1.1
X-Request-Id: REQUEST-02
{"source":"codex-extension","logger":{"name":"IpcRouter"},
"codex.app_session_id":"SESSION-03",...}
HTTP/1.1 202 Accepted
{"ok": true, "error": null}
```

Excerpt A5: Codex response-channel handshake excerpt.

```
GET https://chatgpt.com:443/backend-api/codex/responses HTTP/1.1
Connection: Upgrade
[request headers redacted]
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
[response headers redacted]
```

These excerpts are representative anchor reproductions that illustrate the bounded signal classes discussed in Section V.

B. Support-Only Derivative Content Inventory

This subsection consolidates the derived semantic families, cross-layer signals, derivative-set asymmetries, and GPT-side route differentiation that inform Section V. These tables remain interpretive support material only.

Tables C2–C4 collect the cross-layer, derivative-asymmetry, and GPT-route details omitted from Section V for readability.

The separate original-wrapper plus decoded-export support corpus and the two orientation PDFs remain ancillary interpretation aids only. They corroborate recurring skill-document families and wrapper-class interpretation, but neither source class becomes direct evidence for Section V.

TABLE C3
DERIVED STRUCTURAL SHAPE OF THE FOUR CURSOR(AUTO) DERIVATIVE SETS, USED ONLY AS SUPPORT MATERIAL.

Support derivative set	Salient structural facts	Recovered classes	Bounded use in the case-study section
`Cursor Test1` BidiAppend	2,155 raw files and 138 decoded files with 11 `OK` and 127 `GZIP_FAIL`; `ZLIB_FAIL`; overlapping or sliding windows with recurring skill and settings tails.	Workload prompt, recurring skill families, description-like blocks, settings-guide text, plan paths, code fragments, and path classes.	Indicates that the mini-Celery run preserves broad semantic range despite transport damage and non-documentary file boundaries.
`Cursor Test2` BidiAppend	1,866 raw files with 1,723 under `8KB`, only two at or above `500KB`, and two decoded aggregates.	Netprof prompt, recurring skill text, settings-guide material, and mixed path traces.	Indicates a highly asymmetric fragment-size regime that still preserves readable semantic aggregates.
`Cursor Test1` RunSSE	11 raw and 11 decoded files; first file preserves system prompt and workload; heaviest body reaches 45,606 frames and 1,543 text blocks; one file concentrates 23 `AskQuestion` hits; 124 assistant and 70 tool messages in the heaviest body.	System or user fragments, reasoning titles, plan references, backend/path anchors, tool-result messages, and report-writing requests.	Indicates a planning-to-reporting session shape with concentrated question-asking and tool turns.
`Cursor Test2` RunSSE	2 raw and 2 decoded files; the first preserves planning, task definition, and architecture, and the second preserves the main execution body with 78,759 frames, 184 assistant, 170 tool, and 92 reasoning entries.	System prompt, netprof plan frontmatter, architecture overview, implementation or fix traces, and verification flow.	Indicates the strongest two-stage session split in the Cursor derivative sets.

TABLE C4
DERIVED GPT-SIDE ROUTE TAXONOMY, USED ONLY AS SUPPORT MATERIAL.

Derived route	Salient support-visible facts	Bounded reading in the case-study section
`GPT 5.4 chatgpt.com`	Startup burst across `accounts/check`, `codex/models`, `tasks/list`, `usage`, and `environments`; `accounts/check` returns `plan_type: free` while `usage` returns `plan_type: pro`; `codex/models` is a large outlier that exposes `base_instructions`; `tasks/list` exposes task-card metadata; `environments` exposes workspace or repository setup metadata.	Support-only indication of a broader GPT-side control-plane preparation path before or alongside response-channel setup, with multiple account, environment, task, and model views.
`GPT 5.4 chat.openai.com`	Telemetry intake framed as `codex-extension`; 1,689 logged messages dominated by 1,620 warnings; repeated `IpcClient` warnings for `thread-stream-state-changed`; pseudo-URL fetch failures; local DB lock or slow-insert messages; PowerShell snapshot limit; `skills/list` metadata.	Support-only indication that the GPT IDE extension exports runtime, persistence, and resolver telemetry well beyond prompt or completion content.
`GPT 5.4 ab.chatgpt.com`	Three `/v1/initialize`, four `/v1/rgstr`, and thirty-two `/otlp/v1/metrics` rows; feature gates in the mid-30s plus 16 dynamic configs and 6 layer configs; gate-exposure and product-event registration; websocket lifecycle families; `tool = shell_command`; `sandbox_policy = danger-full-access`; shell-snapshot, token-usage, and model metrics.	Support-only indication that experimentation and metrics surfaces capture IDE-extension feature exposure, runtime measurement, and internal event registration.
`GPT(Codex 5.3) IDE Ext. Test1 chatgpt.com`	134 `POST /backend-api/codex/responses` rows with `200 OK`; one `503` followed by recovery; `response.function_call_arguments.delta` appears 72,424 times; generated filenames include `docker-compose.yml`, `mini_celery/models.py`, and `architecture_report.md`; `accounts/check` and `usage` again diverge on plan view.	Support-only indication that the Codex 5.3 route is generation-heavy, file-creation oriented, and more directly exposes streamed output arguments than the GPT 5.4 polling-heavy route.

C. Supplemental ancillary surface table

This subsection records ancillary surface families that co-occurred with the primary control surfaces so the appendix can separate background coexistence traffic from the direct-evidence core used in Section V.

TABLE C5
SUPPLEMENTAL ANCILLARY SURFACE FAMILIES OBSERVED ALONGSIDE THE PRIMARY CONTROL SURFACES.

Ancillary class	Observed runs	Observed raw rows	Representative request path	Bounded interpretation
Extension marketplace preflight	4	20	`OPTIONS /.../extensionquery`	Coexisting marketplace preflight only.
Python package metadata	1	1	`GET setuptools wheel metadata`	Isolated package retrieval only.
IDE telemetry event	4	55	`POST /OneCollector/1.0`	Coexisting IDE telemetry with bounded count fields only.

The table is therefore contextual only: it documents coexisting traffic classes that were reviewed, but it is not used to promote ancillary families into direct E3/E4/E6 evidence.

D. Sanitized environment metadata excerpt (artifact-derived)

This subsection records only high-level host-context metadata needed to interpret the measurement environment.

Listing 1. Sanitized environment metadata excerpt (artifact-derived)

```
Field                       Value
Platform identifier          win32
```

```
OS family                       Windows (exact build redacted)
Working directory               <REDACTED_PATH>
Shell family                    PowerShell-compatible
Timezone                        <REDACTED>
```

This metadata remains framing information rather than action-level telemetry; it helps contextualize the corpus environment without being treated as proof of tool execution, policy behavior, or complete architecture.

E. Minimal JSON examples (spec-compliant illustrations)

In Listings 2–4, dotted canonical field families from Tables VII–VIII are rendered as illustrative JSON projections for readability; the listings do not define a second naming convention.

Listing 2. Example E4 tool-invocation event sharing correlation keys (MCP-path example)

```
{
  "timestamp": "<ISO8601>",
  "event_type": "E4_TOOL_CALL",
  "agent_id": "spiffe://example.org/agent/ide-runtime",
  "owner": "user:<redacted>",
  "session_id": "sess-8f2c...",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "tool": {
    "name": "replay_traffic",
    "server_id": "mcp:proxymock",
    "action": "call",
    "input_digest": "hmac-sha256:<...>"
  },
  "digest_alg": "hmac-sha256",
  "digest_key_ref": "kms:key-v3",
  "redaction": { "applied": true, "rule_id": "scrub-secrets-v2", "secret_findings_count": 1 },
  "outcome": "allowed"
}
```

Listing 3. Example E6 policy-decision event sharing correlation keys

```
{
  "timestamp": "<ISO8601>",
  "event_type": "E6_POLICY_DECISION",
  "agent_id": "spiffe://example.org/agent/ide-runtime",
  "owner": "user:<redacted>",
  "session_id": "sess-8f2c...",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "7f6a5d4c3b2a1908",
  "policy_id": "policy:tool-replay",
  "decision_id": "dec-19c1...",
  "decision": "block",
  "rationale_ref": "policy-trace:dec-19c1..."
}
```

If the deployment includes a separate human approval workflow, an additional approval-extension event would emit `approval.ref`, `approval.actor`, and `approval.phase`, joined by the same trace/request keys.

Listing 4. Example E7 network egress event sharing correlation keys

```
{
  "timestamp": "<ISO8601>",
  "event_type": "E7_EGRESS",
  "agent_id": "spiffe://example.org/agent/ide-runtime",
  "owner": "user:<redacted>",
  "session_id": "sess-8f2c...",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "7f6a5d4c3b2a1908",
  "dst": { "addr": "203.0.113.10", "port": 443, "fqdn": "api.example.com" },
  "bytes_out": 18240,
  "bytes_in": 9001
}
```

Listing 5. Example T6 evidence-control checkpoint record (cross-cutting)

```
{
  "timestamp": "<ISO8601>",
  "event_type": "EVIDENCE_CHECKPOINT",
```

```
  "evidence": {
    "checkpoint_id": "chk-2026-02-28T12:00Z",
    "chain_hash": "sha256:<current-batch-hash>",
    "prev_hash": "sha256:<previous-batch-hash>",
    "hash_alg": "sha256",
    "signature_ref": "kms-signature://keyring/evidence/v1#sig-8842"
  },
  "retention": { "policy_id": "ret-365d-worm-v2" },
  "access": { "audit_stream_ref": "audit://siem/evidence-access/v1" }
}
```

APPENDIX D
DEPLOYMENT TAXONOMY

This appendix is a reader guide to the deployment lens used in the main text. Along the axes of tool-execution location (TE), inference-engine location/control (IE), inference/tool routing (`R_infer`/`R_tool`, abbreviated below as `Ri`/`Rt`), and asset-authorization vectors $\texttt{Auth} = (A_C, A_L, A_T)$, we organize broad categories (BP1–BP5) and detailed scenarios (S1–S13).

TABLE D1
SUMMARY OF BP1–BP5 (DEPLOYMENT CATEGORIES AND PRIMARY OBSERVABILITY BREAKS).

BP	Situation (one sentence)	What breaks most at the enterprise boundary
BP1	Inference and tools complete within the endpoint, so there is no network.	Network observation itself.
BP2	Inference moves to local/nearby/unauthorized devices or bypasses the observability zone.	Observing the existence and meaning of inference.
BP3	Inference and execution are under enterprise control (on-premises/dedicated tenant).	Network-only is insufficient (server-side logs must be designed).
BP4	Tools are internal, but inference relies on external commercial LLMs.	Meaning and policy decisions inside inference.
BP5	Both inference and execution complete in SaaS/external runtimes.	Action semantics (what was read/written) at the enterprise boundary.

Table D1 provides the main-text abstraction. Table D2 then expands that abstraction into scenario-level detail for appendix use.

TABLE D2
DEPLOYMENT SCENARIO MATRIX FOR S1–S13 (SUMMARY).

ID	Deployment scenario	TE/IE	Ri/Rt	Auth	Key gap
S1	Single endpoint, offline local agent/LLM	TE-L / IE-L	None / None	(A,A,A) or (U,U,U)	No network observation
S2	Authorized air-gapped workstation	TE-L / IE-L	None / None	(A,A,A)	Physical media ingress
S3	Authorized endpoint local inference + internal system access	TE-L / IE-L	None / Intranet	(A,A,A)	Inference context unobservable
S4	Shadow BYOD local inference + VPN/internal system access	TE-L / IE-L	None / Intranet/PL	(U,U,U)	Unmanaged endpoint internals
S5	Unauthorized personal inference connected to intranet	TE-L / IE-Rogue	Internet / Intranet	(A,U,A)	Encryption/multiplexing hide semantics
S6	Out-of-band inference bypassing enterprise observation	TE-L / IE-Rogue	OOB / Intranet	(A,U,A)	Inference outside observability zone
S7	Central internal LLM cluster (on-prem inference)	TE-L / IE-I	Intranet / Intranet	(A,A,A)	Requires server-side instrumentation
S8	Enterprise cloud inference via dedicated line/tunnel (PrivateLink)	TE-L / IE-EC	PL / Intranet	(A,A,A)	Must control Ri boundary segment
S9	Execution at a central tool gateway (enterprise mediation)	TE-E / IE-I or EC	Intranet/PL / Intranet/PL	(A,A,A)	Local context needs separate coverage
S10	Server-side agent runtime (headless/swarm on enterprise K8s/IaaS)	TE-E / IE-I or EC	Intranet/PL / Intranet/PL	(A,A,A)	M2M identity and resource governance
S11	External commercial inference via public API direct (client-side tool effects)	TE-L / IE-3P	Internet / (local tools)	(A,3P,A)	Inference meaning/policy external
S12	External inference with enterprise mediation (tool gate)	TE-E / IE-3P	Internet or PL / Intranet	(A,3P,A)	Side effects internal; inference external
S13	Fully delegated SaaS runtime (3P inference + 3P tool execution)	TE-3P / IE-3P	Internet / Internet	(A,3P,3P)	Action semantics move inside vendor

Read as a companion to Table D2, the scenario rows illustrate a stable pattern across BP1–BP5. BP1 shifts evidentiary dependence almost entirely to endpoint or physical-control records; BP2 captures cases in which inference moves to unmanaged, rogue, or out-of-band assets; BP3 retains enterprise control only if server-side telemetry and workload identity are deliberately designed; BP4 preserves some governance over tool-side effects even when inference depends on an external model; and BP5 moves both action semantics and much of the evidentiary trail into third-party infrastructure. Browser piggybacking is best read as an overlay on these scenarios rather than as a separate BP class, because it reuses an existing deployment path while weakening the enterprise's ability to distinguish human and agent activity.

These appendix tables and the interpretive note above support the BP lens used in Section III, particularly in Section III-C.

APPENDIX E
SCOPE-RELATIVE QUALITATIVE FAILURE-MODE ANALYSIS (AXIS-REMOVAL NON-SUBSTITUTABILITY CHECK)

This appendix reports a scope-relative qualitative failure-mode analysis (axis-removal non-substitutability check) for the minimum practical requirements (T1–T6). For each row, one axis is removed while all other assumptions and available signals are held fixed to identify where reconstruction fails in practice. This is not a restatement of axis/question definitions; it tests failure points in the reconstruction procedure under encryption/multiplexing and boundary constraints.

The results are presented as scoped engineering evidence under the BP1–BP5 lens and incident assumptions (Section II-C), not as a universal minimality proof. For reader convenience, BP1–BP5 denote local/no-network, out-of-band bypass, enterprise-controlled, external-inference hybrid, and fully delegated scenarios, respectively.

Appendix E operates at axis level, whereas Appendix A projects the same non-substitutability logic to field-family level for RQ2. Accordingly, low-recurrence fields in Table A2 are not claimed to be empirically recurrent on their own; they are promoted to REQUIRED only when the scoped failure modes below show that no weaker substitute preserves the relevant forensic question.

How to read Table E1: Each row removes one axis, identifies which forensic question becomes unanswerable or materially weaker, tests representative substitutes, and records why those substitutes do not preserve the same evidentiary confidence.

TABLE E1
SCOPE-RELATIVE NON-SUBSTITUTABILITY CHECK: EFFECT OF REMOVING ONE AXIS (T1–T6) ON FORENSIC TASK ANSWERABILITY.

Removed axis	Failing forensic task(s)	Concrete failure mode	Substitute candidates and why they fail under assumptions	Example incident/scenario reference
T1	FQ1, FQ4, FQ5	Egress and policy events may exist, but they cannot be evidentially bound to a unique agent/owner/session, so accountable attribution and action-linked impact analysis fail.	Candidates: device/IP and coarse user identifiers. Failure: these identifiers are unstable under shared runtimes, NAT, and concurrent sessions; without joinable actor/session identity, request-level accountability is not defensible.	DLP-triggered investigation with disputed responsible workload (BP2/BP5).
T2	FQ2, FQ4, FQ5	Side effects and decisions are observable, but the action step that produced each effect cannot be reconstructed, and policy outcomes cannot be tied to the triggering operation.	Candidates: model I/O logs, egress logs, and proxy error outcomes. Failure: they provide before/after snapshots only; under multiplexed concurrency they cannot reconstruct deterministic action-level causality without tool-call events.	Tool-mediated side effect reconstruction (adversarial steering or non-adversarial mis-execution) in gateway-centered operation (BP3/BP4).
T3	FQ2, FQ3, FQ5	Actions are visible, but source context lineage and, where classified, available sensitivity context remain unresolved, making contamination source and exposure scope attribution materially ambiguous.	Candidates: prompt/output snapshots, semantic similarity, and retrieval hit summaries. Failure: they cannot replace explicit provenance links for audit-ready lineage when multiple retrieval/memory artifacts contribute to one action.	Context-contamination incident (indirect prompt injection or memory poisoning).
T4	FQ2, FQ5	Session and tool traces can be built, but external destination, transfer volume, and timing cannot be allocated to action-level effects, so blast-radius and severity triage are ambiguous.	Candidates: NetFlow/IPFIX counters and 5-tuple plus timestamp alignment. Failure: encryption/multiplexing prevents deterministic request-level byte allocation without joinable egress events and timestamps.	Exfiltration-scope triage for external model/runtime paths (BP4/BP5).
T5	FQ1, FQ4	Actions and egress can be observed, but the policy-decision path and decision authority cannot be demonstrated relative to action time. Where a human approval workflow exists, approval status and approval timing likewise cannot be demonstrated.	Candidates: proxy deny logs, HTTP status/error codes, DLP alerts, and narrative approval claims. Failure: silent pass-through and out-of-band approvals are observationally confounded without explicit policy-decision events and, where applicable, explicit approval-workflow events.	High-risk tool execution requiring policy-path evidence (and approval evidence where such a workflow exists).
T6	FQ6 (and downstream trust in FQ1–FQ5)	Reconstruction may be produced, but evidentiary admissibility collapses because integrity, retention, and access-audit guarantees are unverifiable.	Candidates: cross-source consistency checks and redundant collectors. Failure: they cannot replace tamper-evident records, retention controls, and access-audit provenance; coordinated edits can preserve superficial consistency while breaking evidentiary trust.	Post-incident dispute over log tamper-ing/retention/access history.

Table E1 is used only to justify scope-relative REQUIRED-field promotion for the forensic questions in this study. It does not claim universal minimality beyond the stated evidence and deployment assumptions.

References

[1] E. Rescorla, *The transport layer security (tls) protocol version 1.3,* RFC 8446, Accessed: 2026-02-26, 2018. DOI: 10.17487/RFC8446. https://www.rfc-editor.org/info/rfc8446.

[2] M. Thomson and C. Benfield, *Http/2,* RFC 9113, Accessed: 2026-02-26, 2022. DOI: 10.17487/RFC9113. https://www.rfc-editor.org/info/rfc9113.

[3] J. Iyengar and M. Thomson, *Quic: A udp-based multiplexed and secure transport,* RFC 9000, Accessed: 2026-02-26, 2021. DOI: 10.17487/RFC9000. https://www.rfc-editor.org/info/rfc9000.

[4] M. Bishop, *Http/3,* RFC 9114, Accessed: 2026-02-26, 2022. DOI: 10.17487/RFC9114. https://www.rfc-editor.org/info/rfc9114.

[5] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, *Tls encrypted client hello,* RFC 9849, Published: 2026-03; Accessed: 2026-03-23, 2026. DOI: 10.17487/RFC9849. https://www.rfc-editor.org/info/rfc9849.

[6] OWASP GenAI Security Project, *Owasp top 10 for llm applications 2025 (version 2025),* Online, Released: 2024-11-17; Accessed: 2026-02-26, 2024. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/.

[7] OWASP GenAI Security Project, *Owasp top 10 for agentic applications 2026 (version 2026),* Online, Released: 2025-12-09; Accessed: 2026-02-26, 2025. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/.

[8] National Institute of Standards and Technology (NIST), *Artificial intelligence risk management framework (ai rmf 1.0),* NIST AI 100-1, Accessed: 2026-02-26, 2023. DOI: 10.6028/NIST.AI.100-1. https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf.

[9] SPIFFE, *Spiffe id specification,* Online documentation, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://spiffe.io/docs/latest/spiffe-specs/spiffe-id/.

[10] SPIFFE, *Spire: Spiffe runtime environment,* Online documentation, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://spiffe.io/docs/latest/spire-about/.

[11] SPIFFE, *Spiffe overview,* Online documentation, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://spiffe.io/docs/latest/spiffe-about/overview/.

[12] N. Hardy, "The confused deputy (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988, Accessed: 2026-03-01. DOI: 10.1145/54289.871709. https://doi.org/10.1145/54289.871709.

[13] Y. Liu et al., *Prompt injection attack against LLM-integrated applications,* arXiv:2306.05499v3, Version: v3; Updated: 2025-12-29; Accessed: 2026-03-01, 2023. DOI: 10.48550/arXiv.2306.05499. https://arxiv.org/abs/2306.05499.

[14] D. Ruck and M. Sutton, *Indirect prompt injection: Generative ai's greatest security flaw,* Online, Published: 2024-11-01; Accessed: 2026-03-01, 2024. https://cetas.turing.ac.uk/publications/indirect-prompt-injection-generative-ais-greatest-security-flaw.

[15] G. De Stefano, L. Schoenherr, and G. Pellegrino, *Rag and roll: An end-to-end evaluation of indirect prompt manipulations in LLM-based application frameworks,* arXiv:2408.05025v2, Version: v2; Updated: 2024-08-12; Accessed: 2026-03-01, 2024. DOI: 10.48550/arXiv.2408.05025. https://arxiv.org/abs/2408.05025.

[16] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, *Agentpoison: Red-teaming LLM agents via poisoning memory or knowledge bases,* arXiv:2407.12784v1, Version: v1; Updated: 2024-07-17; Accessed: 2026-03-01, 2024. DOI: 10.48550/arXiv.2407.12784. https://arxiv.org/abs/2407.12784.

[17] National Cyber Security Centre (NCSC), *Prompt injection is not sql injection (it may be worse),* Online PDF, Accessed: 2026-03-02, 2025. https://www.ncsc.gov.uk/pdfs/blog-post/prompt-injection-is-not-sql-injection.pdf.

[18] World Wide Web Consortium (W3C), *Trace context,* W3C Recommendation, Published: 2021-11-23; Accessed: 2026-03-01, 2021. https://www.w3.org/TR/2021/REC-trace-context-1-20211123/.

[19] OpenTelemetry, *Logs,* Online specification, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://opentelemetry.io/docs/specs/otel/logs/.

[20] Model Context Protocol, *Security best practices,* Online documentation, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices.

[21] Model Context Protocol, *Authorization,* Online specification, Protocol Revision: 2025-11-25; Accessed: 2026-03-01, 2025. https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization.

[22] Portkey, *Observability (mcp gateway),* Online, Accessed: 2026-02-26, 2026. https://portkey.ai/docs/product/mcp-gateway/observability.

[23] Model Context Protocol, *Model context protocol (mcp) documentation,* Online, Snapshot used in this study: 2026-02-26; Accessed: 2026-02-26, 2026. https://modelcontextprotocol.io/.

[24] mitmproxy, *How mitmproxy works,* Online, Accessed: 2026-02-26, 2026. https://docs.mitmproxy.org/stable/concepts/how-mitmproxy-works/.

[25] Google Cloud, *Model armor overview,* Online, Last updated: 2026-02-25 UTC; Accessed: 2026-03-01, 2026. https://docs.cloud.google.com/model-armor/overview.
[26] Microsoft, *Prompt shields in azure ai content safety,* Online, Last updated: 2026-02-26; Accessed: 2026-03-01, 2026. https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection.
[27] Portkey, *Mcp gateway,* Online, Last modified: 2026-01-28; Accessed: 2026-03-01, 2026. https://portkey.ai/docs/product/mcp-gateway.
[28] Palo Alto Networks, *Detect mcp threats,* Online, Updated: 2026-02-19; Accessed: 2026-03-01, 2026. https://docs.paloaltonetworks.com/ai-runtime-security/administration/prevent-network-security-threats/detect-mcp-threats.
[29] Cloudflare, *Data loss prevention (dlp),* Online, Last updated: 2025-10-22; Accessed: 2026-03-01, 2025. https://developers.cloudflare.com/ai-gateway/features/dlp/.
[30] Microsoft, *Microsoft purview data security and compliance protections for microsoft 365 copilot and other generative ai apps,* Online documentation, Last updated: 2025-12-15; Accessed: 2026-03-01, 2025. https://learn.microsoft.com/en-us/purview/ai-microsoft-purview.
[31] OpenAI, *Tracing,* Online documentation, Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://openai.github.io/openai-agents-python/tracing/.
[32] Anthropic, *Introducing the model context protocol,* Online, Accessed: 2026-02-26, 2024. https://www.anthropic.com/news/model-context-protocol.
[33] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, *Best current practice for oauth 2.0 security,* RFC 9700, Accessed: 2026-02-26, 2025. DOI: 10.17487/RFC9700. https://www.rfc-editor.org/info/rfc9700.
[34] Kong Inc., *Ai mcp oauth2 plugin,* Online documentation, Accessed: 2026-03-01. https://developer.konghq.com/plugins/ai-mcp-oauth2/.
[35] LiteLLM, *All settings,* Online documentation, Accessed: 2026-03-01. https://docs.litellm.ai/docs/proxy/config_settings.
[36] Traceloop, *Openllmetry,* Online, Accessed: 2026-02-26, 2026. https://github.com/traceloop/openllmetry.
[37] Amazon Web Services, *Detect and filter harmful content by using amazon bedrock guardrails,* Online documentation, Accessed: 2026-03-01. https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html.
[38] Microsoft, *Manage audit log retention policies,* Online, Accessed: 2026-02-26, 2026. https://learn.microsoft.com/en-us/purview/audit-log-retention-policies.
[39] OpenTelemetry, *Generative ai semantic conventions (attribute registry),* Online specification, SemConv path observed: 1.40.0 registry track; Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://opentelemetry.io/docs/specs/semconv/registry/attributes/gen-ai/.
[40] OpenTelemetry, *Generative ai semantic conventions (metrics),* Online specification, Document status: Development; Snapshot used in this study: 2026-03-01; Accessed: 2026-03-01. https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-metrics/.
[41] Amazon Web Services, *Cloudtrail log file validation,* Online documentation, Accessed: 2026-03-01. https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html.
[42] Joint Task Force, *Security and privacy controls for information systems and organizations,* NIST SP 800-53 Rev. 5, Accessed: 2026-02-27, 2020. DOI: 10.6028/NIST.SP.800-53r5. https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final.
[43] L. Moreau and P. Missier, *Prov-dm: The prov data model,* W3C Recommendation, Published: 2013-04-30; Accessed: 2026-03-01, 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/.
[44] B. H. Sigelman et al., *Dapper, a large-scale distributed systems tracing infrastructure,* Google Technical Report, Accessed: 2026-02-27, 2010. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/.
[45] T. Pasquier et al., *Practical whole-system provenance capture,* arXiv:1711.05296, Accessed: 2026-02-27, 2017. DOI: 10.48550/arXiv.1711.05296. https://arxiv.org/abs/1711.05296.
[46] T. Pasquier et al., *Runtime analysis of whole-system provenance,* arXiv:1808.06049, Accessed: 2026-02-27, 2018. DOI: 10.48550/arXiv.1808.06049. https://arxiv.org/abs/1808.06049.
[47] Cloud Native Computing Foundation, *Cloudevents specification v1.0.2,* Online, Accessed: 2026-02-27, 2022. https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md.
[48] OpenTelemetry, *Session attributes,* Online specification, Semantic Conventions version: 1.40.0 (released 2026-02-19); Accessed: 2026-03-01, 2026. https://opentelemetry.io/docs/specs/semconv/registry/attributes/session/.
[49] JSON-RPC Working Group, *Json-rpc 2.0 specification,* Online specification, Origin Date: 2010-03-26; Updated: 2013-01-04; Accessed: 2026-03-01, 2010. https://www.jsonrpc.org/specification.
[50] OpenID Foundation, *Openid connect core 1.0 incorporating errata set 2,* Online, Accessed: 2026-02-27, 2023. https://openid.net/specs/openid-connect-core-1_0.html.

[51] OASIS, *Extensible access control markup language (xacml) version 3.0 core specification,* OASIS Standard, OASIS Standard date: 2013-01-22; Accessed: 2026-03-01, 2013. https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.pdf.

[52] Open Policy Agent, *Decision logs,* Online documentation, Accessed: 2026-03-01. https://www.openpolicyagent.org/docs/management-decision-logs.

[53] OpenTelemetry, *Semantic conventions: Network attributes,* Online specification, Semantic Conventions version: 1.40.0 (released 2026-02-19); Accessed: 2026-03-01, 2026. https://opentelemetry.io/docs/specs/semconv/registry/attributes/network/.

[54] P. Hoffman and K. Fujiwara, *Dns terminology,* BCP 219, RFC 9499, Published: 2024-03; Accessed: 2026-03-01, 2024. DOI: 10.17487/RFC9499. https://www.rfc-editor.org/info/rfc9499.

[55] Internet Assigned Numbers Authority, *Ip flow information export (ipfix) information elements registry,* Online Registry, Created: 2007-05-10; Last Updated: 2025-11-24; Accessed: 2026-03-01, 2025. https://www.iana.org/assignments/ipfix/ipfix.xhtml.

[56] D. C. M. Initiative, *Dcmi metadata terms,* Online, Date Issued: 2020-01-20; Accessed: 2026-03-01, 2020. https://www.dublincore.org/specifications/dublin-core/dcmi-terms/.

[57] T. Berners-Lee, R. Fielding, and L. Masinter, *Uniform resource identifier (uri): Generic syntax,* RFC 3986, Accessed: 2026-02-27, 2005. DOI: 10.17487/RFC3986. https://www.rfc-editor.org/info/rfc3986.

[58] R. Polli and L. Pardue, *Digest fields,* RFC 9530, Accessed: 2026-02-27, 2024. DOI: 10.17487/RFC9530. https://www.rfc-editor.org/info/rfc9530.

[59] OpenTelemetry, *Generative ai semantic conventions (spans),* Online, Accessed: 2026-02-27, 2026. https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/.

[60] J. Bradley, H. Tschofenig, T. Lodderstedt, and M. Jones, *Oauth 2.0 token exchange,* RFC 8693, Accessed: 2026-03-05, 2020. DOI: 10.17487/RFC8693. https://www.rfc-editor.org/info/rfc8693.

[61] Google Cloud, *Configure logging,* Online documentation, Accessed: 2026-03-06, 2026. https://docs.cloud.google.com/model-armor/configure-logging.

[62] Microsoft, *Quickstart: Detect prompt attacks with prompt shields,* Online documentation, Accessed: 2026-03-06, 2026. https://learn.microsoft.com/en-us/azure/ai-services/content-safety/quickstart-jailbreak.

[63] Amazon Web Services, *Applyguardrail - amazon bedrock,* Online documentation, Accessed: 2026-03-06, 2026. https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ApplyGuardrail.html.

[64] Amazon Web Services, *How tokens are counted in amazon bedrock,* Online documentation, Accessed: 2026-03-06, 2026. https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html.

[65] Cloudflare, *Logging (ai gateway),* Online documentation, Accessed: 2026-03-06, 2026. https://developers.cloudflare.com/ai-gateway/observability/logging/.

[66] mitmproxy, *Certificates,* Online documentation, Accessed: 2026-03-06, 2026. https://docs.mitmproxy.org/stable/concepts/certificates/.

[67] mitmproxy, *Ignoring domains,* Online documentation, Accessed: 2026-03-06, 2026. https://docs.mitmproxy.org/stable/howto/ignore-domains/.